

JavaScript ৩০ দিনের লার্নিং রুটিন

প্রতিদিনের টপিক · প্র্যাকটিস টাস্ক · রিয়েল প্রজেক্ট

ভিত্তি থেকে DOM পর্যন্ত — এক মাসে সম্পূর্ণ JavaScript

১৫০+ টপিক · ২১০টি প্র্যাকটিস টাস্ক · ৩০টি প্রজেক্ট · ৫টি ক্যাপস্টোন

পর্ব	কী শিখবে
সপ্তাহ ১ · দিন ১-৭	ভিত্তি: সিনটাক্স, ভেরিয়েবল, অপারেটর, কন্ডিশন, লুপ, স্ট্রিং
সপ্তাহ ২ · দিন ৮-১৪	সংখ্যা, Math, BigInt, ফাংশন, স্কোপ, ক্লোজার, অবজেক্ট
সপ্তাহ ৩ · দিন ১৫-২১	Constructor, Destructuring, Array, Set, Map, Generator
সপ্তাহ ৪ · দিন ২২-২৮	Date, Temporal, Type, RegExp, DOM ও Events
শেষ ধাপ · দিন ২৯-৩০	Error, Debugging, Best Practices, HTML-First, ক্যাপস্টোন

দৈনিক সময়: সকাল ১০:০০-১:০০ (শেখা ও প্র্যাকটিস) + দুপুর ২:০০-২:৩০ (প্রজেক্ট শেষ, কমিট, ডিপ্লয়)

এই রুটিন কীভাবে ব্যবহার করবে

- ▶ প্রতিদিনের কাঠামো একই: আগে টপিক পড়া ও কোড করে দেখা, তারপর প্র্যাকটিস টাস্ক, শেষে দিনের প্রজেক্ট। তিনটির কোনোটি বাদ দিলে পরের দিনে ফাঁক থেকে যাবে।
- ▶ শুধু পড়ে যেও না — প্রতিটি কনসেপ্ট নিজের হাতে অন্তত একবার লিখে চালাও। W3Schools/MDN থেকে কোড কপি করে নয়, দেখে বুঝে নিজে টাইপ করে।
- ▶ প্র্যাকটিস টাস্কগুলোর উত্তর আগে কাগজে অনুমান করো, তারপর কোড চালিয়ে মিলিয়ে দেখো। অনুমান ভুল হলে সেখানেই আসল শেখা হয়।
- ▶ প্রতিদিনের প্রজেক্ট ছোট রাখো — নিখুঁত নয়, চালু হওয়াই লক্ষ্য। প্রথম দিকে UI ছাড়া কনসোলে করলেও চলবে; দিন ২৬ থেকে DOM শিখে সবগুলো UI-তে রূপ দেবে।
- ▶ প্রতিদিন GitHub-এ কমিট করো: js-30-days রিপোর ভেতরে day-01, day-02 ... ফোল্ডার। ৩০ দিনে একটি পূর্ণ পোর্টফোলিও তৈরি হয়ে যাবে।
- ▶ প্রতিদিন শেষে ৫ মিনিটে তিনটি লাইন লেখো: আজ কী শিখলাম, কোথায় আটকেছি, কাল কী পুনরায় দেখব।
- ▶ আটকে গেলে ২০ মিনিটের বেশি একা লড়বে না — কনসোলের error বার্তা পড়ো, MDN দেখো, তারপর এগিয়ে যাও এবং রিভিশন দিনে ফিরে এসো।
- ▶ দিন ৭, ১৪, ২১ ও ২৮ রিভিশন ও ক্যাপস্টোনের দিন — নতুন কিছু শিখবে না, আগের সব একসাথে ব্যবহার করবে।

দৈনিক সময়-ছক

সময়	কাজ	লক্ষ্য
৪:০০ – ৮:০০	কনসেপ্ট নিজের হাতে কোড করা	কনসেপ্ট বোঝা, নোট নেওয়া
৮:১৫ – ৯:৩০	বিরতি	চোখ ও মস্তিষ্কে বিশ্রাম
১০:০০ – ১২:০০	দিনের টপিক পড়া ও ভিডিও দেখা	প্রতিটি উদাহরণ নিজে টাইপ করে চালানো
১২:০০ – ১২:৩০	বিরতি	নোট দ্রুত চোখ বুলানো
১২:৩৫ – ০১:১৫	প্র্যাকটিস টাস্ক সমাধান	৭টি টাস্কের অন্তত ৫টি শেষ
২:০০ – ৩:০০	দিনের প্রজেক্ট শুরু	মূল ফিচার দাঁড় করানো
৩:০০ – ৩:৩৫	প্রজেক্ট শেষ, README, কমিট, ডিপ্লয়	GitHub-এ দিনের কাজ জমা

রিভিশন সাইকেল — যা শিখেছ তা যেন ভুলে না যাও

- ▶ নতুন টপিক শেখার ১ দিন, ৩ দিন, ৭ দিন ও ২১ দিন পর — এই চারবার সংক্ষেপে ফিরে দেখো (spaced repetition)।
- ▶ প্রতিদিন কাজ শুরুর আগে আগের দিনের চিট-শিটে ৫ মিনিট চোখ বুলাও।
- ▶ প্রতি রিভিশন দিনে (৭, ১৪, ২১) আগের একটি প্রজেক্ট না দেখে আবার লেখো — এটিই সবচেয়ে কার্যকর পরীক্ষা।
- ▶ প্রতিটি সপ্তাহ শেষে নিজের হাতে এক পাতার চিট-শিট লেখো; ৩০ দিন শেষে সেগুলো মিলিয়ে নিজের রেফারেন্স বই হয়ে যাবে।

৩০ দিনের মানচিত্র — এক নজরে

দিন	যা শিখবে	দিনের প্রজেক্ট
১	JavaScript পরিচিতি, সেটআপ ও আউটপুট	My Digital ID Card
২	Variables ও Data Types	Personal Data Type Explorer
৩	Operators, Booleans ও Logical Operators	Smart Calculator Engine
৪	Conditionals — if, else, Ternary ও Switch	ATM Menu Simulator
৫	Loops — for, while, break, continue	Multiplication Table & Pattern Generator
৬	Strings — Templates, Methods ও Search	Text Analyzer & Formatter
৭	সপ্তাহ ১ রিভিশন ও ক্যাপস্টোন	Console Quiz Game (সপ্তাহ ১ ক্যাপস্টোন)
৮	Numbers, Math Object ও Random	Random Toolkit — Password, Dice & Lottery
৯	Bitwise Operations ও BigInt	Role & Permission Flag System
১০	Functions — Parameters, Return ও Arguments	Reusable Utility Library (utils.js)
১১	Function Expressions, Arrow Functions ও Timers	Countdown, Clock & Stopwatch (3-in-1)
১২	Scope, Hoisting, Closure ও Strict Mode	Private Counter & Secure Wallet Module
১৩	Objects — Properties, Methods ও this	Course Catalog Explorer
১৪	সপ্তাহ ২ রিভিশন ও ক্যাপস্টোন	Student Result Processing System (সপ্তাহ ২ ক্যাপস্টোন)
১৫	Object Constructors, Destructuring ও Spread	Student Enrollment System
১৬	Arrays — Creation, Methods ও Array Const	Shopping Cart Engine
১৭	Array Search ও Array Sort	Product Filter & Leaderboard
১৮	Array Iterations — map, filter, reduce	Sales & Performance Dashboard (ডেটা লেয়ার)
১৯	Sets, Set Logic ও WeakSet	Tag Manager & Skill Matcher
২০	Maps, WeakMap, Iterators ও Generators	Attendance Register + ID Generator

২১	সপ্তাহ ৩ রিভিশন ও ক্যাপস্টোন	Expense Tracker (সপ্তাহ ৩ ক্যাপস্টোন)
২২	Dates — Formats, Get ও Set Methods	Age Counter & Bangla Date Formatter
২৩	Temporal API — আধুনিক তারিখ ও সময়	Multi-Timezone Class Scheduler
২৪	Data Types গভীরে — Symbols, NaN, Coercion	Type Inspector Playground
২৫	Regular Expressions — সম্পূর্ণ	Form Validator Regex Library
২৬	DOM পরিচিতি ও এলিমেন্ট নির্বাচন	DOM Explorer Tool
২৭	DOM পরিবর্তন, CSS ও Animation	Dynamic Card Builder + Theme Switcher
২৮	Events — Listener, Bubbling ও Delegation	Interactive To-Do App (সপ্তাহ ৪ ক্যাপস্টোন)
২৯	Errors, Debugging ও Best Practices	Bug Hunt & Code Quality Audit
৩০	Form Validation, HTML-First ও ফাইনাল ক্যাপস্টোন	ফাইনাল ক্যাপস্টোন — Personal Learning Dashboard

সপ্তাহ ১ · দিন ১-৭

JavaScript-এর ভিত্তি

দিন ১ · JavaScript পরিচিতি, সেটআপ ও আউটপুট

আজকের টপিক

JavaScript Introduction · JavaScript Where To · JavaScript Output · JavaScript Syntax · JavaScript Statements · JavaScript Comments

মূল কনসেপ্ট

- ▶ JavaScript কী — ব্রাউজারের ভেতরে চলা স্ক্রিপ্টিং ভাষা; V8 ইঞ্জিন কীভাবে কোড পড়ে ও চালায়
- ▶ কোথায় লিখবে — inline attribute, internal `<script>`, external .js ফাইল; পেশাদার কাজে সবসময় external
- ▶ `<script src="app.js">` কোথায় বসাবে — `</body>`-এর ঠিক আগে, নাকি `<head>`-এ defer দিয়ে
- ▶ defer বনাম async — কোনটি HTML পার্সিং আটকায় না এবং কোনটি ক্রম ধরে রাখে
- ▶ আউটপুটের ৫ পথ — `innerHTML`, `textContent`, `document.write()`, `window.alert()`, `console.log()`; কোনটি কখন
- ▶ Syntax-এর উপাদান — value, literal, variable, operator, expression, keyword, identifier; JS case-sensitive
- ▶ Statement ও semicolon — ASI (Automatic Semicolon Insertion) কোথায় বিপদ ডেকে আনে
- ▶ Code block {}, ইন্ডেন্টেশন ও লাইনের দৈর্ঘ্য (৮০ অক্ষরের প্রচলিত নিয়ম)
- ▶ Comment — `//` এক লাইন, `/* */` একাধিক লাইন; কোড বাদ দিতে ও ফাইল-হেডার লিখতে ব্যবহার

প্র্যাকটিস টাস্ক

1. index.html ও app.js বানিয়ে external স্ক্রিপ্ট যুক্ত করো; কনসোলে নিজের নাম প্রিন্ট করো।
2. একই বার্তা ৫টি ভিন্ন পদ্ধতিতে আউটপুট দাও এবং প্রতিটির পাশে কমেন্টে সুবিধা-অসুবিধা লেখো।
3. `<head>`-এ স্ক্রিপ্ট রেখে একবার defer ছাড়া, একবার defer দিয়ে চালাও — কোন ক্ষেত্রে `getElementById()` null দেয় লিখে রাখো।
8. ইচ্ছে করে ৫টি syntax ভুল করো (`Console.log`, কোট না বন্ধ করা, বাংলা কোট চিহ্ন) — কনসোলের error বার্তা পড়ে বাংলায় অর্থ লেখো।
৫. পেজ লোড হওয়ার পর `document.write()` চালালে কী ঘটে পরীক্ষা করে ফলাফল কমেন্টে লেখো।

- ৬. একটি ফাইল-হেডার কমেন্ট ব্লক লেখো (ফাইলের নাম, লেখক, তারিখ, উদ্দেশ্য) এবং সেটিকে টেমপ্লেট হিসেবে সংরক্ষণ করো।
- ৭. `console.log`, `console.table`, `console.warn`, `console.error` — চারটিই ব্যবহার করে পার্থক্য দেখো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: My Digital ID Card

একটি একপাতার পেজ যেখানে তোমার পরিচয়-কার্ড পুরোটাই JavaScript দিয়ে তৈরি হবে — HTML-এ শুধু খালি কনটেইনার থাকবে, সব লেখা JS থেকে বসবে।

- ▶ ☐ external `app.js` থেকে নাম, পেশা, শহর ও এক লাইনের স্লোগান `innerHTML` দিয়ে বসানো
- ▶ ☐ একটি ফিল্ড `textContent` দিয়ে বসিয়ে `innerHTML`-এর সঙ্গে পার্থক্য কমেণ্টে নোট করা
- ▶ ☐ পেজ লোড হলে কনসোলে একটি স্বাগত ব্যানার ও লোড-টাইম প্রিন্ট
- ▶ ☐ পুরো ফাইলের ওপরে হেডার কমেন্ট এবং প্রতিটি সেকশনে ব্যাখ্যামূলক কমেণ্ট
- ▶ ☐ GitHub-এ `js-30-days` রিপো খুলে `day-01` ফোল্ডারে কমিট করা

দিন ২ · Variables ও Data Types

আজকের টপিক

JavaScript Variables · JavaScript Let · JavaScript Const · JavaScript Datatypes

মূল কনসেপ্ট

- ▶ ভেরিয়েবল ঘোষণার ৪ পথ: var, let, const এবং কিছুই না (implicit global — কখনোই নয়)
- ▶ নামকরণের নিয়ম: অক্ষর/_/\$ দিয়ে শুরু, সংখ্যা দিয়ে নয়, reserved word নয়, camelCase রীতি
- ▶ let — block scope, পুনরায় ঘোষণা করা যায় না, মান বদলানো যায়
- ▶ const — ঘোষণার সময়ই মান দিতে হয়, রেফারেন্স বদলানো যায় না, কিন্তু object/array-এর ভেতরের মান বদলানো যায়
- ▶ var — function scope, hoisting-এ undefined, পুরনো কোডে দেখবে কিন্তু নতুন কোডে এড়াবে
- ▶ ৮টি ডেটা টাইপ: String, Number, BigInt, Boolean, Undefined, Null, Symbol, Object
- ▶ Primitive বনাম Reference — কপি করলে কী হয়, === কীভাবে তুলনা করে
- ▶ typeof অপারেটর এবং typeof null === "object" — ঐতিহাসিক বাগ
- ▶ Dynamic typing — একই ভেরিয়েবলে ভিন্ন টাইপ রাখা যায়, কেন এতে সাবধান থাকতে হয়

প্র্যাকটিস টাস্ক

১. var, let, const দিয়ে একই ভেরিয়েবল দুইবার ঘোষণা করার চেষ্টা করো — প্রতিটির error বার্তা টুকে রাখো।
২. const দিয়ে একটি object ও array বানিয়ে ভেতরের মান বদলাও; তারপর পুরো object পুনরায় assign করার চেষ্টা করো।
৩. ৮টি ডেটা টাইপের প্রতিটির জন্য একটি করে ভেরিয়েবল বানিয়ে typeof দিয়ে টাইপ প্রিন্ট করো।
৪. undefined ও null — তিনটি করে পার্থক্য কোড উদাহরণসহ লেখো।
৫. একটি primitive ও একটি object কপি করে মূলটির মান বদলাও — কোনটিতে প্রভাব পড়ল ব্যাখ্যা করো।
৬. ১০টি ভেরিয়েবল নাম লেখো — ৫টি বৈধ, ৫টি অবৈধ; অবৈধগুলোর কারণ কমেণ্টে লেখো।
৭. let x = 5; x = "পাঁচ"; — dynamic typing দেখাও এবং কেন এটি বাগের কারণ হয় লেখো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Personal Data Type Explorer

একটি পেজ যেখানে তোমার নিজের প্রোফাইল ডেটা (নাম, বয়স, ছাত্র কি না, দক্ষতার তালিকা, ঠিকানা object, বড় সংখ্যা BigInt) রেখে প্রতিটির টাইপসহ টেবিল আকারে দেখানো হবে।

- ▶ ☐ অন্তত ৭টি ভিন্ন ডেটা টাইপের ভেরিয়েবল, প্রতিটি সঠিক const/let দিয়ে ঘোষিত

- ▶ ☐ প্রতিটি মানের পাশে typeof ফলাফল টেবিলের আলাদা কলামে
- ▶ ☐ একটি const object-এর ভেতরের ফিল্ড বদলে দেখানো যে reference অপরিবর্তিত থাকে
- ▶ ☐ console.table() দিয়ে একই ডেটা কনসোলে ছাপানো
- ▶ ☐ README-তে "let vs const vs var" তুলনা-টেবিল লিখে কমিট

দিন ৩ · Operators, Booleans ও Logical Operators

আজকের টপিক

JavaScript Operators · JavaScript Arithmetic · JavaScript Assignment · JavaScript Comparison ·
JavaScript Booleans · JavaScript Logical Operators

মূল কনসেপ্ট

- ▶ Arithmetic: + - * / % ** ++ --; pre-increment বনাম post-increment
- ▶ % (remainder)-এর ব্যবহারিক প্রয়োগ — জোড়/বিজোড়, চক্রাকার ইনডেক্স, প্রতি n-তম ধাপ
- ▶ Assignment: = += -= *= /= %= **= এবং logical assignment ||= &&= ??=
- ▶ Comparison: == বনাম ===, != বনাম !== — কেন সবসময় === ব্যবহার করবে
- ▶ > < >= <= এবং string তুলনায় Unicode ক্রমের প্রভাব
- ▶ ৮টি falsy মান: false, 0, -0, 0n, "", null, undefined, NaN — বাকি সবই truthy
- ▶ Logical && || ! এবং short-circuit; && ও || আসলে boolean নয়, মান ফেরত দেয়
- ▶ Nullish coalescing ?? বনাম || — 0 ও খালি string-এর ক্ষেত্রে পার্থক্য
- ▶ Operator precedence: ** > * / % > + - > তুলনা > && > || > =
- ▶ + অপারেটরের দ্বৈত চরিত্র — যোগ বনাম string concatenation

প্র্যাকটিস টাস্ক

1. 10 + "5", 10 - "5", "10" * 2, true + 1, null + 1, undefined + 1 — প্রতিটির ফল আগে অনুমান করো, তারপর চালিয়ে মিলিয়ে দেখো।
2. let i = 5; নিয়ে i++ ও ++i-এর পার্থক্য console.log দিয়ে প্রমাণ করো।
3. ৮টি falsy মানের প্রতিটিকে একটি লুপে if()-এ বসিয়ে যাচাই করো।
4. == ও === দিয়ে ১০ জোড়া মান তুলনা করে একটি ফলাফল টেবিল বানাও।
5. 0 || "default" এবং 0 ?? "default" — পার্থক্য কোডে দেখিয়ে ব্যাখ্যা লেখো।
6. && ও || short-circuit ব্যবহার করে অপ্রয়োজনীয় ফাংশন-কল আটকানোর উদাহরণ লেখো।
7. এমন একটি এক্সপ্রেশন লেখো যেখানে বন্ধনী না দিলে precedence-এর কারণে ভুল ফল আসে; তারপর বন্ধনী দিয়ে ঠিক করো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Smart Calculator Engine

একটি ক্যালকুলেটরের হিসাব-ইঞ্জিন (এখনো UI নয়) — দুটি সংখ্যা ও একটি অপারেটর নিয়ে সব ধরনের হিসাব ও তুলনার ফল দেখাবে।

- ▶ ☐ সাতটি arithmetic ফল (+ - × ÷ % ** এবং পূর্ণসংখ্যা ভাগফল) একসঙ্গে প্রদর্শন
- ▶ ☐ compound assignment (+=, *= ইত্যাদি) দিয়ে চলমান ফলাফল বা running total
- ▶ ☐ দুটি সংখ্যার জন্য পূর্ণ comparison টেবিল (>, <, ==, ===, !=, !==)
- ▶ ☐ শূন্য দিয়ে ভাগ ও অ-সংখ্যা ইনপুটের জন্য ?? দিয়ে নিরাপদ ডিফল্ট মান
- ▶ ☐ জোড়/বিজোড় ও ধনাত্মক/ঋণাত্মক নির্ণয় logical operator দিয়ে

দিন ৪ · Conditionals — if, else, Ternary ও Switch

আজকের টপিক

JavaScript Conditionals · JavaScript if · JavaScript else · The JavaScript Ternary Operator · JavaScript Switch Statement · JavaScript Control Flow

মূল কনসেপ্ট

- ▶ if, if...else, if...else if...else — শর্তের ক্রম কেন ফলাফল বদলে দেয়
- ▶ একক লাইনের if এবং ব্রেস না দেওয়ার বিপদ
- ▶ Nested if বনাম early return / guard clause — কোনটি বেশি পাঠযোগ্য
- ▶ Ternary condition ? a : b — কখন পড়তে সহজ, কখন এড়াতে হবে
- ▶ Chained ternary এবং তার পাঠযোগ্যতার সীমা
- ▶ switch — case, break, default; switch সবসময় === দিয়ে তুলনা করে
- ▶ Fall-through — ইচ্ছাকৃত (case গ্রুপ করা) বনাম অনিচ্ছাকৃত বাগ
- ▶ if বনাম switch — কোন পরিস্থিতিতে কোনটি বেছে নেবে
- ▶ Lookup object দিয়ে switch প্রতিস্থাপন — আধুনিক ও পরিচ্ছন্ন প্যাটার্ন
- ▶ Control flow — কোড কোন পথে যায় তা কাগজে ট্রেস করে বোঝা

প্র্যাকটিস টাস্ক

১. মার্কস নিয়ে A+/A/B/C/F গ্রেড বের করার if...else if চেইন লেখো।
২. একই লজিক ternary chain দিয়ে লিখে দুটির পাঠযোগ্যতা তুলনা করো।
৩. switch দিয়ে ১-৭ সংখ্যা থেকে বাংলা বারের নাম বের করো, default-এ ভুল ইনপুটের বার্তা দাও।
৪. break মুছে ফেলে fall-through দেখো; তারপর ইচ্ছাকৃত গ্রুপিং (শুক্র-শনি → ছুটি) লেখো।
৫. Guard clause ব্যবহার করে ৩ স্তরের nested if-কে সমতল করো।
৬. একটি lookup object বানিয়ে switch ছাড়াই একই কাজ করো।
৭. লগইন যাচাই লেখো: ইউজারনেম খালি → এক বার্তা, পাসওয়ার্ড ছোট → আরেক বার্তা, দুটোই ঠিক → সফল।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: ATM Menu Simulator

কনসোল-ভিত্তিক একটি এটিএম — ব্যালান্স দেখা, টাকা জমা, টাকা তোলা, পিন বদল ও বেরিয়ে যাওয়ার মেনু; সব সিদ্ধান্ত if ও switch দিয়ে।

- ▶ ☐ switch দিয়ে ৫টি মেনু অপশনের রাউটিং

- ▶ ☐ পিন যাচাই — ৩ বার ভুল হলে কার্ড ব্লক; nested if ও guard clause-এর ব্যবহার
- ▶ ☐ টাকা তোলার সময় ব্যালান্স যথেষ্ট কি না ও ১০০-এর গুণিতক কি না যাচাই
- ▶ ☐ ternary দিয়ে প্রতিটি লেনদেনের পর সংক্ষিপ্ত স্ট্যাটাস বার্তা
- ▶ ☐ প্রতিটি লেনদেন কনসোলে লগ এবং শেষে পূর্ণ সারসংক্ষেপ

দিন ৫ · Loops — for, while, break, continue

আজকের টপিক

JavaScript Loops · JavaScript For Loop · JavaScript While Loops · JavaScript Break · JavaScript Continue

মূল কনসেপ্ট

- ▶ for লুপের তিন অংশ: initialization, condition, update — প্রতিটির ভূমিকা
- ▶ for...of — iterable-এর মান ধরে; for...in — key/index ধরে (array-তে ব্যবহার করা ঝুঁকিপূর্ণ)
- ▶ while ও do...while — শর্ত আগে যাচাই না করে, অন্তত একবার চলার নিশ্চয়তা
- ▶ break — লুপ ভাঙা; continue — চলতি চক্র বাদ দিয়ে পরেরটিতে যাওয়া
- ▶ Labeled break/continue — nested লুপ থেকে একবারে বেরিয়ে আসা
- ▶ Infinite loop কীভাবে তৈরি হয় এবং কীভাবে এড়াতে
- ▶ Nested loop — নামতা, প্যাটার্ন ও ম্যাট্রিক্স তৈরির ভিত্তি
- ▶ লুপের ভেতর let বনাম var — closure-এর ক্লাসিক ফাঁদ
- ▶ লুপের কর্মক্ষমতা: arr.length ক্যাশ করা, অপ্রয়োজনীয় কাজ লুপের বাইরে রাখা

প্র্যাকটিস টাস্ক

১. ১ থেকে ১০০ পর্যন্ত সংখ্যার যোগফল তিনভাবে (for, while, do...while) বের করো।
২. FizzBuzz লেখো — ৩-এর গুণিতকে Fizz, ৫-এ Buzz, ১৫-তে FizzBuzz, ১ থেকে ১০০ পর্যন্ত।
৩. একটি অ্যারের ওপর for, for...of ও for...in — তিনভাবে ঘুরে পার্থক্য নোট করো।
৪. break দিয়ে প্রথম মৌলিক সংখ্যা পাওয়া মাত্র লুপ থামাও।
৫. continue দিয়ে ঋণাত্মক সংখ্যাগুলো বাদ দিয়ে বাকিগুলোর গড় বের করো।
৬. Nested লুপ ও labeled break দিয়ে ২ডি অ্যারেতে একটি মান খুঁজে বেরিয়ে আসো।
৭. for (var i...) ও for (let i...)-এর ভেতর setTimeout বসিয়ে ফলাফলের পার্থক্য দেখো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Multiplication Table & Pattern Generator

ব্যবহারকারীর দেওয়া সংখ্যা অনুযায়ী নামতা, তারকা-প্যাটার্ন ও পিরামিড তৈরি করে পেজে দেখানোর টুল।

- ▶ ☐ ১ থেকে n পর্যন্ত যেকোনো সংখ্যার নামতা nested লুপ দিয়ে
- ▶ ☐ তিন ধরনের প্যাটার্ন: ডান-ত্রিভুজ, উল্টো ত্রিভুজ ও পিরামিড
- ▶ ☐ ১-n পর্যন্ত মৌলিক সংখ্যা ও ফিবোনাচি সিরিজ আলাদা সেকশনে

- ▶ ☐ break/continue ব্যবহার করে নির্দিষ্ট সংখ্যা বাদ দেওয়ার অপশন
- ▶ ☐ পুরো আউটপুট <pre> ট্যাগে সাজিয়ে দেখানো

দিন ৬ · Strings — Templates, Methods ও Search

আজকের টপিক

JavaScript Strings · JavaScript String Templates · JavaScript String Methods · JavaScript String Search · JavaScript String Reference

মূল কনসেপ্ট

- ▶ String তৈরির ৩ উপায় — single quote, double quote, backtick; কোথায় কোনটি
- ▶ Escape character: `\' \\" \\ \n \t`
- ▶ String immutable — মেথড নতুন string ফেরত দেয়, মূলটি বদলায় না
- ▶ Template literal: `${}` দিয়ে interpolation, multi-line, ভেতরে expression ও nesting
- ▶ কাটাকাটি: `slice()`, `substring()`, `substr()` (পুরনো, এড়িয়ে চলো) — ঋণাত্মক ইনডেক্স কেবল `slice`-এ
- ▶ বদল: `replace()`, `replaceAll()`, `toUpperCase()`, `toLowerCase()`, `trim()`, `trimStart()`, `trimEnd()`, `padStart()`, `padEnd()`, `repeat()`, `concat()`
- ▶ ভাঙা-জোড়া: `split()` ও `join()` — array-এর সাথে সেতু
- ▶ অক্ষর ধরা: `charAt()`, `at()`, `[]`, `charCodeAt()`, `codePointAt()`
- ▶ খোঁজা: `indexOf()`, `lastIndexOf()`, `includes()`, `startsWith()`, `endsWith()`, `search()`, `match()`, `matchAll()`

প্র্যাকটিস টাস্ক

১. একটি বাক্যের প্রথম ও শেষ ৫ অক্ষর `slice()` দিয়ে বের করো (ঋণাত্মক ইনডেক্সসহ)।
২. Template literal দিয়ে ৫ লাইনের একটি ঠিকানা-ব্লক বানাও এবং ভেতরে একটি হিসাব বসাও।
৩. একটি ফাংশন লেখো যা প্রতিটি শব্দের প্রথম অক্ষর বড় করে দেয়।
৪. `split()` + `reverse()` + `join()` দিয়ে বাক্যের শব্দক্রম উল্টাও।
৫. `padStart()` দিয়ে ইনভয়েস নম্বর INV-00007 ফরম্যাটে বানাও।
৬. একটি প্যারাগ্রাফে নির্দিষ্ট শব্দ কতবার আছে `indexOf()` লুপ দিয়ে গোনো।
৭. `replaceAll()` দিয়ে একটি টেমপ্লেট বার্তায় `{name}` ও `{course}` প্লেসহোল্ডার প্রতিস্থাপন করো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Text Analyzer & Formatter

টেক্সটএরিয়ায় লেখা দিলে যে টুল অক্ষর, শব্দ ও বাক্য গুনে পরিসংখ্যান দেখাবে এবং কয়েক রকম ফরম্যাটে রূপান্তর করবে।

- ▶ ☐ অক্ষর সংখ্যা (স্পেসসহ ও স্পেস ছাড়া), শব্দ সংখ্যা, বাক্য সংখ্যা ও পড়ার আনুমানিক সময়

- ▶ ☐ Title Case, UPPER, lower, camelCase ও slug (my-post-title) রূপান্তর
- ▶ ☐ নির্দিষ্ট শব্দ খুঁজে সবগুলো বদলে দেওয়ার find & replace
- ▶ ☐ সবচেয়ে বেশি ব্যবহৃত ৫টি শব্দের তালিকা
- ▶ ☐ template literal দিয়ে সাজানো রিপোর্ট আউটপুট

দিন ৭ · সপ্তাহ ১ রিভিশন ও ক্যাপস্টোন

আজকের টপিক

Day 1-6 পুনরাবৃত্তি · নিজের চিট-শিট তৈরি · কোড রিফ্যাক্টর · সাপ্তাহিক প্রজেক্ট

মূল কনসেপ্ট

- ▶ প্রতিটি দিনের নোট ১৫ মিনিটে দ্রুত পড়ে ভুলে যাওয়া জায়গা চিহ্নিত করা
- ▶ নিজের হাতে চিট-শিট লেখা — অপারেটর, falsy মান, string মেথড, লুপের ধরন
- ▶ আগের ৬ দিনের কোড রিফ্যাক্টর: `var` → `let/const`, `==` → `===`, ম্যাজিক নম্বর সরানো
- ▶ একই সমস্যার দুটি সমাধান লিখে তুলনা করা (যেমন `if-chain` বনাম `lookup object`)
- ▶ ইন্টারভিউ প্রস্তুতি: `truthy/falsy`, `==` বনাম `===`, `template literal`, `break` বনাম `continue`
- ▶ যা শিখেছি তা না দেখে আবার লেখার অভ্যাস (active recall)

প্র্যাকটিস টাস্ক

1. Day 1-6-এর নোট থেকে এক পাতার নিজের চিট-শিট লেখো।
2. আগের সব ফাইলে `var` খুঁজে `let/const`-এ বদলাও।
3. ২০টি ছোট প্রশ্নের কুইজ নিজে বানিয়ে নিজেই উত্তর দাও।
8. Day 3 ও Day 6-এর একটি করে টাস্ক না দেখে আবার লেখো।
৫. প্রতিটি দিনের একটি করে "সবচেয়ে কঠিন লেগেছে" পয়েন্ট লিখে ব্যাখ্যা দাও।
৬. GitHub রিপোর README.md-এ প্রথম সপ্তাহের অগ্রগতি লেখো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Console Quiz Game (সপ্তাহ ১ ক্যাপস্টোন)

১০টি প্রশ্নের একটি কুইজ — প্রশ্ন দেখাবে, উত্তর নেবে, স্কোর গুনবে, গ্রেড দেবে এবং ভুল উত্তরের সঠিক জবাব দেখাবে।

- ▶ ☐ প্রশ্ন ও উত্তর আলাদা অ্যারে/object-এ, লুপ দিয়ে একে একে জিজ্ঞাসা
- ▶ ☐ সঠিক-ভুল গণনা, শতাংশ হিসাব ও `if...else if` দিয়ে গ্রেড নির্ণয়
- ▶ ☐ ভুল উত্তরের তালিকা শেষে সঠিক জবাবসহ দেখানো
- ▶ ☐ `break` দিয়ে "quit" লিখলে যেকোনো সময় বেরিয়ে আসার সুযোগ
- ▶ ☐ `template literal` দিয়ে সাজানো ফল-কার্ড এবং GitHub-এ কমিট

সপ্তাহ ২ · দিন ৮-১৪

সংখ্যা, ফাংশন, স্কোপ ও অবজেক্ট

দিন ৮ · Numbers, Math Object ও Random

আজকের টপিক

JavaScript Numbers · JavaScript Number Methods · JavaScript Number Properties · JavaScript Number Reference · JavaScript Math Object · JavaScript Math Reference · JavaScript Random

মূল কনসেপ্ট

- ▶ JS-এ সব সংখ্যা 64-bit float (IEEE 754) — তাই $0.1 + 0.2 \neq 0.3$
- ▶ নিরাপদ সীমা: `Number.MAX_SAFE_INTEGER`, `MIN_SAFE_INTEGER`, `EPSILON`
- ▶ বিশেষ মান: `Infinity`, `-Infinity`, `NaN`; এবং `NaN !== NaN`
- ▶ রূপান্তর: `Number()`, `parseInt()`, `parseFloat()`, `+str` — কোনটি কীভাবে আচরণ করে
- ▶ ফরম্যাট: `toFixed()`, `toPrecision()`, `toString(2/8/16)`, `toLocaleString('bn-BD')`
- ▶ যাচাই: `Number.isInteger()`, `Number.isFinite()`, `Number.isNaN()` বনাম গ্লোবাল `isNaN()`
- ▶ `Math`: `round`, `ceil`, `floor`, `trunc`, `abs`, `sign`, `pow`, `sqrt`, `cbrt`, `min`, `max`, `hypot`
- ▶ `Math.round()` বনাম `toFixed()` — ঋণাত্মক সংখ্যা ও .5-এ পার্থক্য
- ▶ `Math.random()` — $0 \leq x < 1$; নির্দিষ্ট রেঞ্জের পূর্ণসংখ্যা বের করার সূত্র

প্র্যাকটিস টাস্ক

1. $0.1 + 0.2$ ছেপে দেখো; `toFixed(2)` ও `EPSILON` — দুইভাবে সমাধান করো।
2. `parseInt("12px")`, `Number("12px")`, `parseFloat("3.14 টাকা")` — ফল তুলনা করো।
3. একটি ফাংশন লেখো `randomInt(min, max)` যা দুই প্রান্তসহ পূর্ণসংখ্যা দেয়।
4. `Math.round`, `ceil`, `floor`, `trunc` দিয়ে -2.5 ও 2.5 রূপান্তর করে টেবিল বানাও।
5. `toLocaleString('bn-BD')` দিয়ে ১২৩৪৫৬৭.৮৯ টাকাকে বাংলা মুদ্রা-ফরম্যাটে দেখাও।
6. দশমিক সংখ্যাকে বাইনারি ও হেক্সাডেসিমালে রূপান্তর করো `toString(radix)` দিয়ে।
7. একটি অ্যারের সর্বোচ্চ ও সর্বনিম্ন `Math.max(...arr)` দিয়ে বের করো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Random Toolkit — Password, Dice & Lottery

তিনটি ইউটিলিটি একসাথে — শক্তিশালী পাসওয়ার্ড জেনারেটর, ছক্কা রোলার এবং লটারি নম্বর পিকার।

- ▶ ☐ দৈর্ঘ্য ও অক্ষর-শ্রেণি (বড় হাতের, ছোট হাতের, সংখ্যা, চিহ্ন) বেছে পাসওয়ার্ড তৈরি
- ▶ ☐ দুটি ছক্কা রোল, যোগফল এবং ১০০ বার রোলের পরিসংখ্যান
- ▶ ☐ ১-৪৯ থেকে ৬টি অনন্য লটারি নম্বর, পুনরাবৃত্তি ছাড়া
- ▶ ☐ সব ফলাফল toLocaleString ও toFixed দিয়ে সুন্দর ফরম্যাটে
- ▶ ☐ পাসওয়ার্ডের শক্তি-স্কোর দৈর্ঘ্য ও বৈচিত্র্য অনুযায়ী হিসাব করে দেখানো

দিন ৯ · Bitwise Operations ও BigInt

আজকের টপিক

JavaScript Bitwise Operations · JavaScript BigInt

মূল কনসেপ্ট

- ▶ সংখ্যা বাইনারিতে কেমন দেখায়; bitwise কাজের জন্য JS 32-bit signed integer ব্যবহার করে
- ▶ & (AND), | (OR), ^ (XOR), ~ (NOT), << (বাম শিফট), >> (ডান শিফট), >>> (সাইন ছাড়া শিফট)
- ▶ ব্যবহারিক প্রয়োগ: permission flag, toggle, দ্রুত জোড়/বিজোড় ($n \& 1$), $x \ll 1$ মানে গুণ ২
- ▶ ^ দিয়ে দুটি ভেরিয়েবল swap এবং "একবার আছে এমন সংখ্যা" বের করা
- ▶ 32-bit সীমা পেরোলে কেন ভুল ফল আসে
- ▶ BigInt কেন দরকার — MAX_SAFE_INTEGER-এর ওপরে নির্ভুল হিসাব
- ▶ BigInt লেখা: 123n বা BigInt(123); typeof → "bigint"
- ▶ BigInt ও Number মেশানো যায় না (TypeError), কিন্তু == দিয়ে তুলনা চলে
- ▶ BigInt-এ ভাগফলে দশমিক থাকে না — ফল truncate হয়

প্র্যাকটিস টাস্ক

১. ০-১৫ পর্যন্ত প্রতিটি সংখ্যার বাইনারি রূপ toString(2).padStart(4,'0') দিয়ে ছাপাও।
২. &, |, ^, ~ — চারটির জন্য একটি করে সত্যক-সারণি তৈরি করো।
৩. $n \& 1$ দিয়ে জোড়/বিজোড় নির্ণয় করে $n \% 2$ -এর সাথে ফল ও গতি তুলনা করো।
৪. XOR দিয়ে দুটি ভেরিয়েবল অস্থায়ী ভেরিয়েবল ছাড়া অদলবদল করো।
৫. << ও >> দিয়ে সংখ্যাকে ২ দিয়ে গুণ ও ভাগ করে দেখাও।
৬. BigInt দিয়ে ৩০ ও ৫০-এর ফ্যাক্টোরিয়াল বের করো এবং Number দিয়ে করলে কী হয় দেখো।
৭. BigInt ও Number যোগ করার চেষ্টা করে error পড়ো, তারপর সঠিকভাবে রূপান্তর করো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Role & Permission Flag System

bitwise ফ্লাগ দিয়ে একটি রোল-পারমিশন ব্যবস্থা — Read=1, Write=2, Delete=4, Publish=8, Admin=16 — সাথে BigInt দিয়ে বড় হিসাবের ক্যালকুলেটর।

- ▶ ☐ ৫টি পারমিশন কনস্ট্যান্ট ও ৪টি রোল (Student, Teacher, Manager, Admin) ফ্লাগ যোগ করে তৈরি
- ▶ ☐ hasPermission(role, perm) ফাংশন & দিয়ে যাচাই
- ▶ ☐ পারমিশন যোগ (|), বাদ (& ~) ও টগল (^) — তিনটি আলাদা ফাংশন

- ▶ ☐ প্রতিটি রোলার বাইনারি রূপ ও পারমিশনের তালিকা টেবিলে প্রদর্শন
- ▶ ☐ BigInt দিয়ে ১০০ পর্যন্ত ফ্যাক্টোরিয়াল ও বড় সংখ্যার গুণ দেখানোর আলাদা অংশ

দিন ১০ · Functions — Parameters, Return ও Arguments

আজকের টপিক

JavaScript Functions · Functions are Code Blocks · Invoking JavaScript Functions · JavaScript Function Parameters · JavaScript Function Return · JavaScript Function Arguments

মূল কনসেপ্ট

- ▶ Function declaration ও তার hoisting — ঘোষণার আগেই কল করা যায়
- ▶ Invoking-এর পথ: সরাসরি কল, ইভেন্ট থেকে, অন্য ফাংশন থেকে, নিজে থেকে (IIFE)
- ▶ Parameter বনাম Argument — নাম আর প্রকৃত মান
- ▶ Default parameter: function f(a, b = 10); undefined-এ কাজ করে কিন্তু null-এ নয়
- ▶ Rest parameter ...args — অনির্দিষ্ট সংখ্যক আর্গুমেন্ট ধরা
- ▶ পুরনো arguments object — array-like, arrow function-এ নেই
- ▶ return — একটিই মান ফেরত দেয়; একাধিক দিতে হলে object বা array
- ▶ Early return দিয়ে nested if কমানো
- ▶ একটি ফাংশন = একটি কাজ (Single Responsibility); pure function কী ও কেন ভালো
- ▶ ফাংশনও একধরনের value — ভেরিয়েবলে রাখা ও আর্গুমেন্ট হিসেবে পাঠানো যায়

প্র্যাকটিস টাস্ক

1. calculateArea(shape, ...dims) লেখো যা বৃত্ত, বর্গ ও ত্রিভুজের ক্ষেত্রফল দেয়।
2. Default parameter দিয়ে greet(name = "অতিথি", lang = "bn") লেখো।
3. Rest parameter দিয়ে sum(...nums) ও average(...nums) লেখো।
4. একটি ফাংশনে arguments ব্যবহার করে দেখো, তারপর arrow function-এ চেষ্টা করে error দেখো।
5. একটি ফাংশন লেখো যা object রিটার্ন করে: {min, max, sum, avg}।
6. Early return দিয়ে ব্যবহারকারীর ইনপুট যাচাইয়ের ৪ স্তরের if সমতল করো।
7. একটি pure ও একটি impure ফাংশন লিখে পার্থক্য কमेंটে ব্যাখ্যা করো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Reusable Utility Library (utils.js)

নিজের একটি ছোট ইউটিলিটি লাইব্রেরি, যেটি পরের সব প্রজেক্টে বারবার ব্যবহার করবে।

- ▶ □ অন্তত ১২টি ফাংশন: capitalize, slugify, truncate, randomInt, formatBDT, isEmpty, clamp, sum, average, percentage, range, pluralize

- ▶ ☐ প্রতিটি ফাংশনে default ও rest parameter-এর যথাযথ ব্যবহার
- ▶ ☐ প্রতিটির ওপরে JSDoc-স্টাইল কमेंট — প্যারামিটার ও রিটার্ন টাইপসহ
- ▶ ☐ একটি test.js ফাইলে প্রতিটি ফাংশনের অন্তত ২টি করে পরীক্ষা
- ▶ ☐ GitHub-এ আলাদা ফোল্ডারে রেখে README-তে ব্যবহারবিধি লেখা

দিন ১১ · Function Expressions, Arrow Functions ও Timers

আজকের টপিক

JavaScript Function Expressions · JavaScript Arrow Functions · JavaScript Functions Quiz · JavaScript Timers

মূল কনসেপ্ট

- ▶ Function expression — ভেরিয়েবলে বসানো ফাংশন; hoist হয় না, তাই আগে কল করা যায় না
- ▶ Anonymous function ও named function expression — stack trace-এ নাম কেন কাজে লাগে
- ▶ IIFE: (function(){})(0) — module pattern-এর ভিত্তি
- ▶ Arrow function সিনটাক্স: এক প্যারামিটারে বন্ধনী ঐচ্ছিক, এক লাইনে implicit return
- ▶ Arrow থেকে object রিটার্ন করতে ({...}) বন্ধনী লাগে
- ▶ Arrow-এ নিজস্ব this, arguments, super নেই — lexical this
- ▶ কখন arrow ব্যবহার করবে না: object method, constructor, this-নির্ভর event handler
- ▶ Callback ও higher-order function — ফাংশনকে আর্গুমেন্ট হিসেবে পাঠানো
- ▶ setTimeout / clearTimeout এবং setInterval / clearInterval
- ▶ Timer-এর নির্ভুলতা — নির্দিষ্ট সময়ের "কমপক্ষে" পরে চলে, ঠিক ততক্ষণে নয়; event loop-এর ভূমিকা

প্র্যাকটিস টাস্ক

১. একই কাজের ফাংশন তিনভাবে লেখো (declaration, expression, arrow) — তারপর ঘোষণার আগে কল করে দেখো।
২. একটি IIFE লিখে ভেতরে একটি private কাউন্টার রাখো।
৩. ৫টি সাধারণ ফাংশনকে arrow-এ রূপান্তর করো, implicit return সহ।
৪. একটি object-এর method-এ arrow ব্যবহার করে this ভাঙা দেখাও, তারপর ঠিক করো।
৫. setTimeout দিয়ে ৩ সেকেন্ড পর বার্তা দাও এবং clearTimeout দিয়ে বাতিলের বোতাম যুক্ত করো।
৬. setInterval দিয়ে প্রতি সেকেন্ডে কাউন্টার বাড়াও, ১০-এ পৌঁছে clearInterval করো।
৭. একটি higher-order function repeat(n, callback) লেখো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Countdown, Clock & Stopwatch (3-in-1)

একটি পেজে তিনটি টাইমার-টুল — লাইভ ডিজিটাল ঘড়ি, নির্দিষ্ট তারিখ পর্যন্ত কাউন্টডাউন এবং স্টার্ট/স্টপ/রিসেট স্টপওয়াচ।

- ▶ ☐ setInterval দিয়ে প্রতি সেকেন্ডে আপডেট হওয়া ঘড়ি — ঘণ্টা, মিনিট, সেকেন্ড
- ▶ ☐ কাউন্টডাউনে দিন-ঘণ্টা-মিনিট-সেকেন্ড; শেষ হলে clearInterval ও সমাপ্তি বার্তা
- ▶ ☐ স্টপওয়াচে start / pause / reset — তিনটি আলাদা callback
- ▶ ☐ সব লজিক arrow function ও higher-order function দিয়ে লেখা
- ▶ ☐ সংখ্যার আগে শূন্য বসাতে padStart(2, "0") ব্যবহার

দিন ১২ · Scope, Hoisting, Closure ও Strict Mode

আজকের টপিক

JavaScript Scope · JavaScript Code Blocks · JavaScript Hoisting · JavaScript var let and const · JavaScript Use Strict

মূল কনসেপ্ট

- ▶ তিন ধরনের scope: global, function ও block
- ▶ Lexical scope — ভেতরের ফাংশন বাইরের ভেরিয়েবল দেখতে পায়, উল্টোটা নয়
- ▶ Scope chain — নাম খোঁজার ক্রম ভেতর থেকে বাইরে
- ▶ Closure — ফাংশন তার জন্মস্থানের scope মনে রাখে; সবচেয়ে গুরুত্বপূর্ণ ইন্টারভিউ টপিক
- ▶ Hoisting: var → undefined, let/const → TDZ error, function declaration → পুরোটা ওপরে
- ▶ Temporal Dead Zone কী এবং কেন তৈরি করা হয়েছে
- ▶ Global variable-এর দূষণ এবং window object-এর সাথে সম্পর্ক
- ▶ "use strict" কী বদলায়: undeclared variable নিষিদ্ধ, duplicate parameter নিষিদ্ধ, this undefined
- ▶ IIFE ও module pattern দিয়ে ডেটা সত্যিকারের private রাখা

প্র্যাকটিস টাস্ক

১. তিন স্তরের nested ফাংশন লিখে scope chain ট্রেস করো।
২. var, let, const — তিনটিরই hoisting আচরণ কোডে প্রমাণ করো।
৩. একটি closure-ভিত্তিক createCounter() লেখো যা পরস্পর স্বাধীন কাউন্টার দেয়।
৪. "use strict" দিয়ে ও ছাড়া x = 10 (ঘোষণা ছাড়া) চালিয়ে পার্থক্য দেখো।
৫. লুপে var দিয়ে তৈরি ক্লাসিক closure বাগ দেখাও এবং let দিয়ে ঠিক করো।
৬. Module pattern দিয়ে একটি bankAccount বানাও যার ব্যালান্স বাইরে থেকে বদলানো যায় না।
৭. Block scope প্রমাণ করতে {}-এর ভেতরে let ঘোষণা করে বাইরে ব্যবহার করার চেষ্টা করো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Private Counter & Secure Wallet Module

Closure ও module pattern ব্যবহার করে এমন একটি ওয়ালেট যেখানে ব্যালান্স সরাসরি ছোঁয়া যায় না — কেবল নির্দিষ্ট মেথড দিয়ে বদলানো যায়।

- ▶ ☐ createWallet(owner, initial) → deposit, withdraw, getBalance, history — চারটি মেথড
- ▶ ☐ ব্যালান্স সম্পূর্ণ private — বাইরে থেকে wallet.balance = 999999 কাজ করবে না

- ▶ ☐ প্রতিটি লেনদেনের ইতিহাস closure-এর ভেতরের অ্যারেতে জমা
- ▶ ☐ একই ফ্যাক্টরি থেকে ৩টি স্বাধীন ওয়ালেট বানিয়ে পরস্পর প্রভাবহীনতা প্রমাণ
- ▶ ☐ পুরো ফাইলে "use strict" এবং শূন্যসংখ্যক global variable

দিন ১৩ · Objects — Properties, Methods ও this

আজকের টপিক

JavaScript Objects · JavaScript Object Properties · JavaScript Object Methods · JavaScript this Keyword · JavaScript Display Objects

মূল কনসেপ্ট

- ▶ Object literal — key/value জোড়া; key সবসময় string বা Symbol
- ▶ Dot notation বনাম bracket notation — কখন bracket বাধ্যতামূলক
- ▶ Property যোগ, বদল, মুছে ফেলা (delete) ও অস্তিত্ব যাচাই (in, hasOwnProperty)
- ▶ Nested object ও optional chaining ?. দিয়ে নিরাপদে গভীরে যাওয়া
- ▶ Method — object-এর ভেতরের ফাংশন; ES6 shorthand syntax
- ▶ this — কে কল করল তার ওপর নির্ভর করে; ৪টি প্রেক্ষাপট (method, একা ফাংশন, arrow, event handler)
- ▶ Object.keys(), Object.values(), Object.entries() এবং তাদের ওপর লুপ
- ▶ for...in লুপ এবং inherited property-এর ফাঁদ
- ▶ Object দেখানো: console.log, console.table, JSON.stringify(obj, null, 2)

প্র্যাকটিস টাস্ক

১. ৫ স্তরের nested object বানিয়ে গভীরে থাকা মান dot ও bracket দুইভাবে পড়ো।
২. ?. ব্যবহার করে না-থাকা property পড়ার চেষ্টা করে error এড়াও।
৩. একটি object-এ ৩টি method লিখে this দিয়ে নিজের property ব্যবহার করো।
৪. Object method-এ arrow function দিয়ে this ভাঙা দেখাও, তারপর ঠিক করো।
৫. Object.entries() ও for...of দিয়ে সব key-value ছাপাও।
৬. delete, in ও hasOwnProperty — তিনটির ব্যবহার একটি উদাহরণে দেখাও।
৭. JSON.stringify(obj, null, 2) দিয়ে সুন্দর করে ছেপে ফাইলের মতো দেখাও।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Course Catalog Explorer

একটি অনলাইন কোর্স-ক্যাটালগ — nested object-এ কোর্স, ব্যাচ, শিক্ষক ও সময়সূচি রেখে পেজে কার্ড আকারে দেখানো।

- ▶ ☐ অন্তত ৫টি কোর্সের nested object: কোর্স → ব্যাচ → শিক্ষক → সময়সূচি
- ▶ ☐ প্রতিটি কোর্স object-এ getSummary() method যা this ব্যবহার করে

- ▶ ☐ `Object.entries()` দিয়ে ঘুরে HTML কার্ড তৈরি
- ▶ ☐ না-থাকা তথ্যের জন্য `?.` ও ডিফল্ট মানের ব্যবস্থা
- ▶ ☐ `console.table()` দিয়ে সব কোর্সের সংক্ষিপ্ত তালিকা কনসোলে

দিন ১৪ · সপ্তাহ ২ রিভিশন ও ক্যাপস্টোন

আজকের টপিক

Day 8-13 পুনরাবৃত্তি · ফাংশন ও স্কোপ ঝালাই · utils.js সম্প্রসারণ · সাপ্তাহিক প্রজেক্ট

মূল কনসেপ্ট

- ▶ ফাংশন ও স্কোপের ধারণা একত্রে ঝালাই: parameter, return, closure, this
- ▶ সংখ্যা ও ফরম্যাটিং পুনরাবৃত্তি: toFixed, toLocaleString, Math মেথড
- ▶ নিজের utils.js লাইব্রেরিকে আরও বড় ও পরিচ্ছন্ন করা
- ▶ রিফ্যাক্টর: বারবার লেখা কোড খুঁজে ফাংশনে পরিণত করা
- ▶ ইন্টারভিউ প্রস্তুতি: closure, hoisting, this, arrow বনাম regular function
- ▶ একই কাজ দুইভাবে লিখে কোনটি বেশি পাঠযোগ্য তা বিচার করা

প্র্যাকটিস টাস্ক

1. Day 8-13-এর নোট থেকে দ্বিতীয় সপ্তাহের চিট-শিট লেখো।
2. আগের যেকোনো ৩টি প্রজেক্ট থেকে পুনরাবৃত্ত কোড বের করে utils.js-এ সরাও।
3. "arrow বনাম regular function" — ৭টি পার্থক্যের টেবিল বানাও।
4. Closure ও this নিয়ে ১০টি ইন্টারভিউ প্রশ্ন লিখে উত্তর দাও।
5. Day 11-এর টাইমার প্রজেক্ট না দেখে আবার লেখো।
6. GitHub রিপোতে সপ্তাহ ২-এর সব কোড কমিট ও README আপডেট করো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Student Result Processing System (সপ্তাহ ২ ক্যাপস্টোন)

একাধিক শিক্ষার্থীর নম্বর নিয়ে ফলাফল প্রক্রিয়াকরণ — গড়, গ্রেড, পাস/ফেল, মেধাক্রম ও মার্কশিট তৈরি।

- ▶ ☐ প্রতিটি শিক্ষার্থীর object (নাম, রোল, ৫ বিষয়ের নম্বর) ও তাদের একটি অ্যারে
- ▶ ☐ calculateTotal, calculateGPA, getGrade, getRank — চারটি আলাদা ফাংশন
- ▶ ☐ সর্বোচ্চ, সর্বনিম্ন ও শ্রেণির গড় নম্বর এবং বিষয়ভিত্তিক পাসের হার
- ▶ ☐ প্রতিটি শিক্ষার্থীর জন্য template literal দিয়ে সাজানো মার্কশিট
- ▶ ☐ সব সংখ্যা toFixed(2) ও বাংলা লোকাল ফরম্যাটে প্রদর্শন

সপ্তাহ ৩ · দিন ১৫-২১

অবজেক্ট, অ্যারে ও কালেকশন

দিন ১৫ · Object Constructors, Destructuring ও Spread

আজকের টপিক

JavaScript Object Constructors · JavaScript Destructuring

মূল কনসেপ্ট

- ▶ Constructor function ও new কীওয়ার্ড — new দিলে চার ধাপে কী ঘটে
- ▶ Constructor-এর ভেতরে this; নাম বড় হাতের অক্ষর দিয়ে শুরু করার রীতি
- ▶ Prototype-এ method রাখা কেন স্মৃতি-সাশ্রয়ী
- ▶ কপি করার তিন উপায়: Object.create(), Object.assign(), spread {...obj}
- ▶ Shallow copy বনাম deep copy; structuredClone() দিয়ে গভীর কপি
- ▶ Object.freeze() ও Object.seal() — কোনটি কতটুকু আটকায়
- ▶ Array destructuring: ক্রম, skip, default মান, swap, rest
- ▶ Object destructuring: নাম মিলিয়ে, rename ({a: x}), default, nested
- ▶ ফাংশন প্যারামিটারে destructuring — নামযুক্ত আর্গুমেন্টের সুবিধা
- ▶ Spread দিয়ে array/object মেলানো এবং rest দিয়ে বাকিটা ধরা

প্র্যাকটিস টাস্ক

1. function Student(name, roll) constructor লিখে ৩টি object বানাও।
2. Method-কে Student.prototype-এ সরিয়ে দুইভাবে মেমরি-ব্যবহারের পার্থক্য ব্যাখ্যা করো।
3. Object.assign() ও spread দিয়ে object কপি করে nested মান বদলে shallow copy-র সমস্যা দেখাও।
8. structuredClone() দিয়ে deep copy করে একই পরীক্ষা আবার করো।
৫. একটি অ্যারে থেকে প্রথম দুটি মান ও বাকিগুলো rest দিয়ে আলাদা করো।
৬. nested object destructure করে rename ও default মান দাও।
9. function createUser({name, role = "student", age}) — প্যারামিটার destructuring লেখো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Student Enrollment System

Constructor দিয়ে শিক্ষার্থী তৈরি, destructuring দিয়ে ফর্ম-ডেটা ভেঙে নেওয়া এবং spread দিয়ে রেকর্ড হালনাগাদ করার একটি ভর্তি-ব্যবস্থা।

- ▶ ☐ Student constructor এবং prototype-এ enroll(), getInfo(), updateFee() মেথড
- ▶ ☐ ফর্ম-ডেটা object থেকে destructuring করে নতুন শিক্ষার্থী তৈরি
- ▶ ☐ spread দিয়ে অপরিবর্তনীয় আপডেট — মূল রেকর্ড না বদলে নতুন কপি
- ▶ ☐ Object.freeze() দিয়ে ভর্তি নিশ্চিত হওয়া রেকর্ড লক করা
- ▶ ☐ সব শিক্ষার্থীর তালিকা টেবিল আকারে এবং console.table()-এ

দিন ১৬ · Arrays — Creation, Methods ও Array Const

আজকের টপিক

JavaScript Arrays · The new Array() Constructor · JavaScript Array Methods · JavaScript Array Const · JavaScript Array Reference

মূল কনসেপ্ট

- ▶ Array literal [] বনাম new Array() — new Array(5)-এর বিখ্যাত ফাঁদ
- ▶ Array আসলে object; index হলো string key; তাই typeof নয়, Array.isArray() ব্যবহার করো
- ▶ length — শুধু পড়ার জন্য নয়, লিখেও অ্যাারে ছোট/বড় করা যায়
- ▶ যোগ-বিয়োগ: push, pop, shift, unshift
- ▶ কাটা-জোড়া: slice() কপি ফেরত দেয়, splice() মূল অ্যাারে বদলায়
- ▶ concat, join, reverse, fill, flat, flatMap, at()
- ▶ অপরিবর্তনীয় নতুন মেথড: toSorted, toReversed, toSpliced, with
- ▶ const array — মান যোগ/বাদ চলে, কিন্তু পুরো অ্যাারে পুনরায় assign করা যায় না
- ▶ কপি করার উপায়: slice(), spread [...arr], Array.from()
- ▶ 2D array ও nested array-এর গঠন

প্র্যাকটিস টাস্ক

1. new Array(5) ও new Array(5, 6) — পার্থক্য কোডে দেখাও।
2. slice() ও splice() একই অ্যাারেতে চালিয়ে মূল অ্যাারের কী হলো দেখাও।
3. push/pop/shift/unshift দিয়ে একটি stack ও একটি queue বানাও।
4. flat(Infinity) দিয়ে ৪ স্তরের nested array সমতল করো।
5. const array-তে push করো, তারপর পুরোটা পুনরায় assign করে error দেখো।
6. Array.from({length: 10}, (_, i) => i + 1) দিয়ে ১-১০ তৈরি করো।
7. একটি ৩×৩ ম্যাট্রিক্স বানিয়ে সারি ও কলামের যোগফল বের করো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Shopping Cart Engine

একটি শপিং কার্টের পূর্ণ লজিক — পণ্য যোগ, পরিমাণ বাড়ানো-কমানো, মুছে ফেলা, মোট হিসাব ও কুপন।

- ▶ ☐ addItem, removeItem, updateQty, clearCart — push/splice/findIndex দিয়ে
- ▶ ☐ একই পণ্য দ্বিতীয়বার যোগ করলে পরিমাণ বাড়বে, নতুন সারি তৈরি হবে না

- ▶ ☐ সাবটোটাল, ভ্যাট, ডেলিভারি চার্জ ও গ্র্যান্ড টোটাল হিসাব
- ▶ ☐ কার্ট খালি থাকলে আলাদা বার্তা এবং length দিয়ে ব্যাজ কাউন্ট
- ▶ ☐ slice() দিয়ে কার্টের অপরিবর্তনীয় স্ল্যাপশট সেভ করে "আগের অবস্থায় ফেরা"

দিন ১৭ · Array Search ও Array Sort

আজকের টপিক

JavaScript Array Search · JavaScript Array Sort

মূল কনসেপ্ট

- ▶ মান দিয়ে খোঁজা: `indexOf`, `lastIndexOf`, `includes`
- ▶ শর্ত দিয়ে খোঁজা: `find`, `findIndex`, `findLast`, `findLastIndex`
- ▶ `includes` বনাম `indexOf` — `NaN` খোঁজার ক্ষেত্রে পার্থক্য
- ▶ `sort()`-এর ডিফল্ট আচরণ: সব মানকে `string` ধরে তুলনা → `[1, 10, 2]`
- ▶ Compare function (`a, b`) => `a - b` কীভাবে কাজ করে — ঋণাত্মক, শূন্য, ধনাত্মক
- ▶ সংখ্যায় উর্ধ্ব/অধঃক্রম এবং `string`-এ `localeCompare` (বাংলা ও ইউনিকোডে জরুরি)
- ▶ Object array-কে property অনুযায়ী `sort` করা এবং একাধিক শর্তে `sort`
- ▶ `sort()` মূল অ্যারে বদলায় — `toSorted()` বা কপি করে `sort` করো
- ▶ `reverse()` ও `stable sort`-এর গুরুত্ব
- ▶ Random shuffle — `sort(() => Math.random() - 0.5)` কেন ভুল, Fisher-Yates কেন সঠিক

প্র্যাকটিস টাস্ক

১. একটি সংখ্যার অ্যারেতে `sort()` চালিয়ে ভুল ফল দেখাও, তারপর `compare function` দিয়ে ঠিক করো।
২. `find` ও `filter`-এর পার্থক্য একই ডেটায় দেখাও।
৩. বাংলা নামের অ্যারে `localeCompare('bn')` দিয়ে সাজাও।
৪. Object array-কে দাম অনুযায়ী উর্ধ্বক্রমে ও রেটিং অনুযায়ী অধঃক্রমে `sort` করো।
৫. দুই শর্তে `sort` করো: আগে বিভাগ, তারপর নম্বর।
৬. `[NaN].includes(NaN)` ও `[NaN].indexOf(NaN)` তুলনা করো।
৭. Fisher-Yates অ্যালগরিদমে একটি অ্যারে `shuffle` করো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Product Filter & Leaderboard

২০টি পণ্যের ডেটা নিয়ে খোঁজা, ছাঁকা ও সাজানোর একটি টেবিল-ভিত্তিক টুল।

- ▶ ☐ নাম দিয়ে লাইভ সার্চ (`includes + toLowerCase`)
- ▶ ☐ দাম, রেটিং ও নামের ভিত্তিতে উর্ধ্ব/অধঃক্রম `sort` বোতাম
- ▶ ☐ সর্বোচ্চ রেটিংয়ের ৫টি পণ্য নিয়ে লিডারবোর্ড

- ▶ ☐ "স্টকে নেই" পণ্য findIndex দিয়ে খুঁজে আলাদা চিহ্ন দেওয়া
- ▶ ☐ মূল ডেটা অপরিবর্তিত রেখে toSorted()/কপি দিয়ে সাজানো

দিন ১৮ · Array Iterations — map, filter, reduce

আজকের টপিক

JavaScript Array Iterations · JavaScript Array Reference

মূল কনসেপ্ট

- ▶ forEach — শুধু ঘোরে, কিছু ফেরত দেয় না; break চলে না
- ▶ map — একই দৈর্ঘ্যের নতুন অ্যারে; রূপান্তরের জন্য
- ▶ filter — শর্তে টেকা মান নিয়ে ছোট অ্যারে
- ▶ reduce — সব মান থেকে একটি ফল; accumulator ও initial value কেন জরুরি
- ▶ reduceRight, some, every, flatMap
- ▶ entries(), keys(), values() এবং for...of-এর সাথে ব্যবহার
- ▶ Method chaining: filter().map().reduce() — পাঠযোগ্যতা ও কর্মক্ষমতার ভারসাম্য
- ▶ map বনাম forEach — কখন কোনটি বেছে নেবে
- ▶ reduce দিয়ে গ্রুপিং, গণনা এবং object তৈরি
- ▶ Callback-এর তিন প্যারামিটার: (value, index, array)

প্র্যাকটিস টাস্ক

১. একই কাজ for, forEach ও map — তিনভাবে করে পার্থক্য লেখো।
২. filter দিয়ে পাস করা শিক্ষার্থী, map দিয়ে শুধু নাম, reduce দিয়ে মোট নম্বর বের করো।
৩. reduce দিয়ে একটি অ্যারেকে বিভাগ অনুযায়ী গ্রুপ করে object বানাও।
৪. some ও every দিয়ে "কেউ কি ফেল করেছে" ও "সবাই কি পাস" যাচাই করো।
৫. flatMap দিয়ে প্রতিটি শিক্ষার্থীর বিষয়-তালিকা একত্র করো।
৬. Chaining দিয়ে এক লাইনে: ৫০-এর বেশি নম্বর → ১০% বোনাস → মোট যোগফল।
৭. reduce দিয়ে একটি বাক্যে প্রতিটি অক্ষরের গণনা বের করো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Sales & Performance Dashboard (ডেটা লেয়ার)

এক মাসের বিক্রয় ডেটা নিয়ে সারসংক্ষেপ, শীর্ষ পারফরমার ও বিভাগভিত্তিক বিশ্লেষণ তৈরির ইঞ্জিন।

- ▶ ☐ reduce দিয়ে মোট বিক্রয়, গড় অর্ডার মূল্য ও মোট কমিশন
- ▶ ☐ filter ও sort দিয়ে শীর্ষ ৫ বিক্রেতা এবং সর্বনিম্ন ৩ জন
- ▶ ☐ reduce দিয়ে অঞ্চল ও পণ্য-শ্রেণি অনুযায়ী গ্রুপিং

- ▶ ☐ map দিয়ে প্রতিটি সারি রিপোর্ট-ফরম্যাটে রূপান্তর
- ▶ ☐ some/every দিয়ে লক্ষ্যমাত্রা অর্জনের সতর্কবার্তা

আজকের টপিক

JavaScript Sets · JavaScript Set Methods · JavaScript Set Logic · JavaScript WeakSet · JavaScript Set Reference

মূল কনসেপ্ট

- ▶ `new Set()` — শুধু অনন্য মান রাখে এবং ঢোকানো ক্রম সংরক্ষণ করে
- ▶ `add`, `delete`, `has`, `size`, `clear`; `has()` array-এর `includes`-এর চেয়ে অনেক দ্রুত
- ▶ `Set` ↔ `Array` রূপান্তর: `[...set]`, `Array.from(set)`, `new Set(arr)`
- ▶ ডুপ্লিকেট বাদ দেওয়ার সবচেয়ে সংক্ষিপ্ত উপায়
- ▶ Set logic: union, intersection, difference, `symmetricDifference`, `isSubsetOf`, `isSupersetOf`, `isDisjointFrom`
- ▶ Set-এ object রাখলে reference দিয়ে তুলনা হয়, মান দিয়ে নয়
- ▶ `forEach` ও `for...of` দিয়ে Set-এর ওপর ঘোরা
- ▶ `WeakSet` — শুধু object রাখে, garbage collection-বান্ধব, `size` ও iteration নেই
- ▶ কখন Set আর কখন Array — খোঁজার গতি বনাম ক্রম ও index

প্র্যাকটিস টাস্ক

১. একটি ডুপ্লিকেটভরা অ্যারে থেকে এক লাইনে অনন্য মান বের করো।
২. দুটি Set-এ union, intersection ও difference বের করো — মেথড ও ম্যানুয়াল দুইভাবে।
৩. ১ লাখ মান নিয়ে `Set.has()` ও `Array.includes()`-এর সময় তুলনা করো।
৪. Set-এ দুটি দেখতে একই object যোগ করে দেখো কেন দুটোই থেকে যায়।
৫. `isSubsetOf` দিয়ে যাচাই করো একটি কোর্সের প্রয়োজনীয় দক্ষতা শিক্ষার্থীর আছে কি না।
৬. `WeakSet` দিয়ে "একবারই প্রক্রিয়া হয়েছে" এমন object চিহ্নিত করো।
৭. একটি বাক্য থেকে অনন্য অক্ষর ও অনন্য শব্দ Set দিয়ে বের করো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Tag Manager & Skill Matcher

ব্লগ পোস্টের ট্যাগ ব্যবস্থাপনা ও চাকরির দক্ষতা মেলানোর টুল — পুরোটাই Set দিয়ে।

- ▶ ☐ ট্যাগ যোগ ও মুছে ফেলা, ডুপ্লিকেট স্বয়ংক্রিয়ভাবে বাদ
- ▶ ☐ দুটি পোস্টের সাধারণ ট্যাগ (intersection) ও অনন্য ট্যাগ (difference)

- ▶ ☐ সব পোস্টের ট্যাগ মিলিয়ে ট্যাগ-ক্লাউড (union) ও ব্যবহারের গণনা
- ▶ ☐ চাকরির চাহিদা বনাম প্রার্থীর দক্ষতা — isSubsetOf দিয়ে মিলের শতাংশ
- ▶ ☐ WeakSet দিয়ে ইতিমধ্যে প্রক্রিয়াকৃত পোস্ট চিহ্নিত করা

আজকের টপিক

JavaScript Maps · JavaScript Map Methods · JavaScript WeakMap · JavaScript Map Reference ·
JavaScript Iterables · JavaScript Iterators · JavaScript Generators

মূল কনসেপ্ট

- ▶ `new Map()` — যেকোনো টাইপের key নিতে পারে: object, function, এমনকি NaN
- ▶ `set`, `get`, `has`, `delete`, `size`, `clear`; `set()` chaining করা যায়
- ▶ Map বনাম Object — key-এর টাইপ, ক্রম, size, iteration ও কর্মক্ষমতা
- ▶ `keys()`, `values()`, `entries()` এবং `for...of`-এ destructuring করে পড়া
- ▶ রূপান্তর: `Object.fromEntries(map)`, `new Map(Object.entries(obj))`
- ▶ WeakMap — private ডেটা রাখার প্যাটার্ন ও মেমরি লিক এড়ানো
- ▶ Iterable protocol — `Symbol.iterator` থাকলেই `for...of` চলে
- ▶ Iterator protocol — `next()` → {value, done}
- ▶ Custom iterable object নিজে বানানো
- ▶ Generator function* ও `yield` — থেমে থেমে মান দেওয়া, infinite sequence, lazy evaluation

প্র্যাকটিস টাস্ক

১. একটি Map বানিয়ে object ও function-কে key হিসেবে ব্যবহার করো।
২. Map ও Object-এর ৭টি পার্থক্যের টেবিল বানাও এবং প্রতিটি কোডে প্রমাণ করো।
৩. একটি বাক্যে শব্দের গণনা Map দিয়ে করো।
৪. `Object.fromEntries()` ও `Object.entries()` দিয়ে দুই দিকেই রূপান্তর করো।
৫. WeakMap দিয়ে একটি object-এর private ডেটা রাখো।
৬. Custom iterable বানাও যা `for...of`-এ ১ থেকে n পর্যন্ত দেয়।
৭. Generator লেখো: একটি infinite ID generator ও একটি Fibonacci sequence।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Attendance Register + ID Generator

Map দিয়ে হাজিরা-খাতা এবং generator দিয়ে স্বয়ংক্রিয় ভর্তি-নম্বর তৈরির ব্যবস্থা।

- ▶ ☐ শিক্ষার্থী object-কে key ধরে Map-এ প্রতিদিনের হাজিরা সংরক্ষণ
- ▶ ☐ প্রতিটি শিক্ষার্থীর উপস্থিতির শতাংশ ও সবচেয়ে অনুপস্থিতদের তালিকা

- ▶ ☐ Generator দিয়ে TQA-2026-0001 খাঁচের ক্রমিক আইডি তৈরি
- ▶ ☐ Custom iterable যা for...of-এ শুধু উপস্থিত শিক্ষার্থী দেয়
- ▶ ☐ WeakMap-এ শিক্ষার্থীর গোপন নোট রাখা যা বাইরে থেকে পড়া যায় না

দিন ২১ · সপ্তাহ ৩ রিভিশন ও ক্যাপস্টোন

আজকের টপিক

Day 15-20 পুনরাবৃত্তি · ডেটা স্ট্রাকচার তুলনা · Iteration সিদ্ধান্ত-গাছ · সাপ্তাহিক প্রজেক্ট

মূল কনসেপ্ট

- ▶ Array, Object, Set, Map — কোন সমস্যায় কোন কাঠামো বেছে নেবে
- ▶ Iteration মেথডের সিদ্ধান্ত-গাছ: map / filter / reduce / find / some / every
- ▶ অপরিবর্তনীয়তা (immutability) কেন গুরুত্বপূর্ণ — মূল ডেটা না বদলানোর অভ্যাস
- ▶ Big-O সংক্ষেপে: Array-তে খোঁজা $O(n)$, Set/Map-এ খোঁজা $O(1)$
- ▶ ইন্টারভিউ: map বনাম forEach, slice বনাম splice, Set বনাম Array, Map বনাম Object
- ▶ পুরনো for লুপকে আধুনিক iteration মেথডে রূপান্তরের অভ্যাস

প্র্যাকটিস টাস্ক

1. Day 15-20-এর চিট-শিট লেখো, অন্তত ৪টি তুলনা-টেবিলসহ।
2. Day 18-এর ড্যাশবোর্ড না দেখে আবার লেখো।
3. একই সমস্যা Array, Set ও Map — তিনভাবে সমাধান করে তুলনা করো।
8. ১৫টি ইন্টারভিউ প্রশ্ন লিখে উত্তর দাও।
৫. আগের প্রজেক্টগুলোতে for লুপ খুঁজে iteration মেথডে রূপান্তর করো।
৬. সপ্তাহ ৩-এর সব কোড কমিট ও README আপডেট করো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Expense Tracker (সপ্তাহ ৩ ক্যাপস্টোন)

আয়-ব্যয়ের হিসাব রাখার সম্পূর্ণ লজিক-লেয়ার — যোগ, ছাঁকা, শ্রেণিবিন্যাস, মাসিক সারসংক্ষেপ ও বাজেট সতর্কতা।

- ▶ ☐ লেনদেন object-এর অ্যারে: তারিখ, ধরন, শ্রেণি, পরিমাণ ও নোট
- ▶ ☐ reduce দিয়ে মোট আয়, মোট ব্যয়, বর্তমান ব্যালান্স ও শ্রেণিভিত্তিক যোগফল
- ▶ ☐ Map দিয়ে মাসভিত্তিক সারসংক্ষেপ, Set দিয়ে ব্যবহৃত শ্রেণির তালিকা
- ▶ ☐ শ্রেণিভিত্তিক বাজেট অতিক্রম করলে সতর্কবার্তা
- ▶ ☐ সব লেনদেন টেবিলে sort ও filter সহ; মোট অংক বাংলা মুদ্রা-ফরম্যাটে

সপ্তাহ ৪ · দিন ২২-২৮

সময়, টাইপ, RegExp, DOM ও ইভেন্ট

দিন ২২ · Dates — Formats, Get ও Set Methods

আজকের টপিক

JavaScript Dates · JavaScript Date Formats · JavaScript Get Date Methods · JavaScript Set Date Methods · JavaScript Date Reference

মূল কনসেপ্ট

- ▶ `new Date()` তৈরির ৪ উপায়: খালি, timestamp, date string, আলাদা আর্গুমেন্ট
- ▶ ISO 8601 (2026-07-30), short ও long ফরম্যাট; timezone-এর প্রভাব
- ▶ Get: `getFullYear`, `getMonth` (০ থেকে শুরু!), `getDate`, `getDay`, `getHours`, `getMinutes`, `getTime`
- ▶ UTC সংস্করণ: `getUTCFullYear`, `getUTCHours` ইত্যাদি
- ▶ Set: `setFullYear`, `setMonth`, `setDate`, `setHours` — এবং auto-correction (৩২ তারিখ → পরের মাস)
- ▶ Timestamp — `Date.now()`, দুই তারিখের পার্থক্য মিলিসেকেন্ডে
- ▶ `toLocaleDateString('bn-BD')`, `toLocaleTimeString` এবং `Intl.DateTimeFormat`
- ▶ Date object mutable — `setDate()` মূল object-কেই বদলে দেয়
- ▶ Date-এর পরিচিত ফাঁদ: মাস ০-ইনডেক্স, string parsing-এর অসামঞ্জস্য, timezone গোলমাল

প্র্যাকটিস টাস্ক

১. চারভাবে Date তৈরি করে সবগুলো ছাপাও।
২. আজকের তারিখ থেকে বছর, মাসের সঠিক নাম, দিন ও বারের নাম বের করো।
৩. দুই তারিখের মধ্যে কত দিন পার্থক্য তা বের করো।
৪. `setDate(getDate() + 45)` দিয়ে ৪৫ দিন পরের তারিখ বের করো।
৫. `toLocaleDateString('bn-BD', {...})` দিয়ে "৩০ জুলাই, ২০২৬, বৃহস্পতিবার" ফরম্যাট বানাও।
৬. জন্মতারিখ থেকে বয়স (বছর, মাস, দিন) বের করার ফাংশন লেখো।
৭. একটি ফাংশন লেখো যা যেকোনো তারিখকে DD/MM/YYYY ও YYYY-MM-DD দুইভাবে দেয়।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Age Counter & Bangla Date Formatter

জন্মতারিখ দিলে বিস্তারিত বয়স, পরবর্তী জন্মদিন পর্যন্ত বাকি সময় এবং বাংলা ফরম্যাটে তারিখ দেখানোর টুল।

- ▶ ☐ বয়স বছর-মাস-দিনে এবং মোট দিন, ঘণ্টা ও মিনিটে
- ▶ ☐ পরবর্তী জন্মদিন পর্যন্ত কাউন্টডাউন এবং সেদিন কোন বার পড়বে
- ▶ ☐ যেকোনো তারিখকে ৫টি ফরম্যাটে রূপান্তর: ISO, DD/MM/YYYY, বাংলা লং, সংক্ষিপ্ত ও relative ("৩ দিন আগে")
- ▶ ☐ দুটি তারিখের মধ্যে কর্মদিবস গণনা (শুক্র-শনি বাদে)
- ▶ ☐ মাস o-ইনডেক্সসহ ৫টি সাধারণ ভুল README-তে নোট করা

দিন ২৩ · Temporal API — আধুনিক তারিখ ও সময়

আজকের টপিক

Temporal Intro · Temporal vs Date · PlainDate / PlainTime / PlainDateTime · PlainYearMonth / PlainMonthDay · ZonedDateTime / Instant · Temporal Duration · Temporal Now · Arithmetic / Differences / Compare · Conversion / Display Formats · Temporal Mistakes / Migration / Standards / Reference

মূল কনসেপ্ট

- ▶ Date-এর সমস্যা কেন Temporal দরকার করল: mutable, timezone দুর্বলতা, মাস ০-ইনডেক্স, parsing অসামঞ্জস্য
- ▶ Temporal-এর মূল ধারণা: সব object immutable এবং প্রতিটি টাইপ স্পষ্ট
- ▶ Temporal.Now.plainDateISO(), plainDateTimeISO(), zonedDateTimeISO(), instant()
- ▶ PlainDate (শুধু তারিখ), PlainTime (শুধু সময়), PlainDateTime (দুটো, timezone ছাড়া)
- ▶ PlainYearMonth (বিলিং চক্র) ও PlainMonthDay (জন্মদিন, বার্ষিকী)
- ▶ ZonedDateTime — টাইমজোনসহ সঠিক মুহূর্ত; Instant — টাইমজোন-নিরপেক্ষ টাইমস্ট্যাম্প
- ▶ Duration — সময়ের পরিমাণ; add(), subtract(), until(), since(), round(), total()
- ▶ তুলনা: Temporal.PlainDate.compare() ও equals()
- ▶ রূপান্তর: toPlainDate(), toZonedDateTime(), toInstant() এবং Date ↔ Temporal সেতু
- ▶ প্রদর্শন: toString() ও toLocaleString('bn-BD')
- ▶ সাধারণ ভুল: PlainDateTime-কে সঠিক মুহূর্ত ভাবা, timezone ছাড়া হিসাব করা
- ▶ ISO 8601 ও IANA timezone মান; ব্রাউজারে না থাকলে @js-temporal/polyfill দিয়ে অনুশীলন

প্র্যাকটিস টাস্ক

1. Temporal.Now-এর ৫টি মেথড চালিয়ে আউটপুট তুলনা করো।
2. একই মুহূর্তকে PlainDateTime, ZonedDateTime ও Instant-এ প্রকাশ করে পার্থক্য লেখো।
3. PlainDate.add({months: 3, days: 10}) দিয়ে ভবিষ্যতের তারিখ বের করো।
4. until() দিয়ে দুই তারিখের মধ্যে Duration বের করে বছর-মাস-দিনে ভাগ করো।
5. ঢাকা, লন্ডন ও নিউইয়র্কের বর্তমান সময় ZonedDateTime দিয়ে দেখাও।
6. PlainMonthDay দিয়ে আগামী ১০ বছরের জন্মদিন কোন বার পড়বে বের করো।
7. একটি পুরনো Date-ভিত্তিক ফাংশনকে Temporal-এ রূপান্তর করে দুটির ফল মিলিয়ে দেখো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Multi-Timezone Class Scheduler

বিভিন্ন দেশে থাকা শিক্ষার্থীর জন্য ক্লাসের সময়সূচি — শিক্ষকের ঢাকার সময় থেকে প্রত্যেকের স্থানীয় সময় বের করা।

- ▶ ☐ ঢাকার সময়ে ক্লাস নির্ধারণ করে লন্ডন, নিউইয়র্ক, টরন্টো ও সিডনির স্থানীয় সময় প্রদর্শন
- ▶ ☐ Duration দিয়ে ক্লাসের দৈর্ঘ্য ও শেষ হওয়ার সময় হিসাব
- ▶ ☐ সাপ্তাহিক পুনরাবৃত্তি — পরবর্তী ৮টি ক্লাসের তারিখ ও সময়
- ▶ ☐ "ক্লাস শুরু হতে আর কত বাকি" লাইভ কাউন্টডাউন (Temporal.Now + until)
- ▶ ☐ ডে-লাইট সেভিং বদলালে সময় ঠিক থাকে কি না পরীক্ষা করে নোট লেখা

দিন ২৪ · Data Types গভীরে — Symbols, NaN, Coercion

আজকের টপিক

JavaScript Data Types · JavaScript Primitives · JavaScript Built-In Objects · JavaScript Symbols · JavaScript typeof · JavaScript undefined · JavaScript NaN · JavaScript toString() · JavaScript toLocaleString() · JavaScript Type Coercion · JavaScript Type Conversion

মূল কনসেপ্ট

- ▶ ৭টি primitive + Object; primitive immutable — বদলানো যায় না, নতুন তৈরি হয়
- ▶ Wrapper object ও autoboxing — "abc".toUpperCase() ভেতরে কীভাবে কাজ করে
- ▶ Built-in object: Object, Array, String, Number, Boolean, Date, Math, JSON, RegExp, Map, Set, Promise, Error
- ▶ typeof-এর ৮টি সম্ভাব্য ফল এবং typeof null === "object" বাগের ইতিহাস
- ▶ undefined বনাম null বনাম খালি string বনাম 0 — চারটির ভূমিকা আলাদা
- ▶ NaN কোথা থেকে আসে, NaN !== NaN, isNaN() বনাম Number.isNaN()
- ▶ Symbol — অনন্য key, Symbol.for() রেজিস্ট্রি, well-known symbol (Symbol.iterator, Symbol.toPrimitive)
- ▶ toString() ও valueOf() — object কীভাবে primitive-এ পরিণত হয়
- ▶ toLocaleString() — সংখ্যা, মুদ্রা (bn-BD, BDT), শতাংশ ও তারিখ ফরম্যাটিং
- ▶ Implicit coercion (+, ==, if) বনাম explicit conversion (String(), Number(), Boolean(), parseInt())
- ▶ বিখ্যাত ফাঁদ: [] + {}, [] == false, null >= 0 সত্য কিন্তু null > 0 মিথ্যা

প্র্যাকটিস টাস্ক

১. ৮টি typeof ফলাফলের প্রতিটির জন্য একটি করে উদাহরণ লেখো।
২. null, undefined, NaN, "", 0, [], {} — সাতটির truthy/falsy টেবিল বানাও।
৩. isNaN("abc") ও Number.isNaN("abc") তুলনা করে পার্থক্য ব্যাখ্যা করো।
৪. দুটি Symbol বানিয়ে একই বর্ণনা দিয়েও অসমতা প্রমাণ করো।
৫. একটি object-এ toString() ও valueOf() override করে + ও template literal-এ আচরণ দেখো।
৬. toLocaleString দিয়ে একই সংখ্যা bn-BD, en-US ও ar-EG লোকেলে ছাপাও।
৭. [] + {}, {} + [], [] == false, "5" - 2, "5" + 2 — ফল অনুমান করে যাচাই করো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Type Inspector Playground

যেকোনো ইনপুট দিলে তার টাইপ, রূপান্তর ও তুলনার ফল দেখানোর একটি শেখার টুল।

- ▶ ☐ ইনপুট বক্সে মান দিলে `typeof`, `truthy/falsy`, `Number()`, `String()`, `Boolean()` ফল
- ▶ ☐ দুই মানের জন্য `==` ও `===` তুলনার পাশাপাশি বাংলায় ব্যাখ্যা
- ▶ ☐ বাংলা মুদ্রা, শতাংশ ও বড় সংখ্যার লোকেল-ফরম্যাটার সেকশন
- ▶ ☐ Symbol দিয়ে object-এ লুকানো property যোগ করে `Object.keys()`-এ না আসা দেখানো
- ▶ ☐ ১০টি বিখ্যাত coercion ফাঁদ নিয়ে ইন্টারঅ্যাকটিভ কুইজ

আজকের টপিক

JavaScript RegExp · RegExp Flags · RegExp Character Classes · RegExp Meta Characters · Regular Expression Assertions · Regular Expression Groups · RegExp Quantifiers · JavaScript RegExp Patterns · JavaScript RegExp Objects · RegExp Methods

মূল কনসেপ্ট

- ▶ দুইভাবে লেখা: literal /pattern/flags এবং new RegExp("pattern", "flags")
- ▶ Flags: g (সব), i (কেস-নিরপেক্ষ), m (মাল্টিলাইন), s (dotAll), u (ইউনিকোড), y (sticky)
- ▶ Character class: [abc], [^abc], [a-z], \d \D \w \W \s \S এবং .
- ▶ Meta character ও escape: . * + ? ^ \$ () [] { } | \ /
- ▶ Quantifier: *, +, ?, {n}, {n,}, {n,m}; greedy বনাম lazy (.*?)
- ▶ Anchor ও assertion: ^, \$, \b, \B, lookahead (?=) (!), lookbehind (?<=) (?<!)
- ▶ Group: capturing (), non-capturing (?:), named (?<year>), alternation |, backreference \1
- ▶ মেথড: test(), exec(), match(), matchAll(), search(), replace(), replaceAll(), split()
- ▶ replace()-এ \$1, \$<name> এবং callback ফাংশনের ব্যবহার
- ▶ lastIndex ও g ফ্ল্যাগের ফাঁদ — একই regex বারবার ব্যবহারে ভুল ফল

প্র্যাকটিস টাস্ক

১. বাংলাদেশি মোবাইল নম্বর (01[3-9] + ৮ সংখ্যা) যাচাইয়ের regex লেখো।
২. ইমেইল যাচাইয়ের regex লিখে ১০টি বৈধ ও ১০টি অবৈধ ঠিকানায় পরীক্ষা করো।
৩. পাসওয়ার্ড শক্তি যাচাই — lookahead দিয়ে বড় হাতের, ছোট হাতের, সংখ্যা ও চিহ্ন বাধ্যতামূলক করো।
৪. Named group দিয়ে 2026-07-30 থেকে বছর, মাস ও দিন আলাদা করো।
৫. matchAll() দিয়ে একটি লেখা থেকে সব হ্যাশট্যাগ ও URL বের করো।
৬. replace() callback দিয়ে প্রতিটি শব্দের প্রথম অক্ষর বড় করো।
৭. Greedy ও lazy quantifier দিয়ে HTML ট্যাগ ধরার পার্থক্য দেখাও।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Form Validator Regex Library

বাংলাদেশি ফর্মের জন্য একটি পুনঃব্যবহারযোগ্য যাচাই-লাইব্রেরি।

- ▶ ☐ ৮টি ভ্যালিডেটর: মোবাইল, ইমেইল, NID, পাসপোর্ট, পোস্টকোড, URL, পাসওয়ার্ড ও বাংলা নাম

- ▶ ☐ প্রতিটি ভ্যালিডেটরের জন্য বাংলা বার্তা ও test()-ভিত্তিক boolean ফল
- ▶ ☐ পাসওয়ার্ড শক্তি মিটার (দুর্বল / মাঝারি / শক্তিশালী) lookahead দিয়ে
- ▶ ☐ লেখা থেকে সব ইমেইল, ফোন ও লিংক বের করার এক্সট্রাক্টর
- ▶ ☐ সার্চ-হাইলাইটার: মিলে যাওয়া অংশ <mark> দিয়ে মোড়ানো

দিন ২৬ · DOM পরিচিতি ও এলিমেন্ট নির্বাচন

আজকের টপিক

The HTML DOM · HTML DOM API · Selecting DOM Elements · HTML DOM Document

মূল কনসেপ্ট

- ▶ DOM কী — HTML-এর গাছের মতো object উপস্থাপন; document হলো মূল প্রবেশপথ
- ▶ Node বনাম Element; nodeType এবং parent/child/sibling সম্পর্ক
- ▶ পুরনো নির্বাচন: getElementById, getElementsByClassName, getElementsByTagName, getElementsByName
- ▶ আধুনিক নির্বাচন: querySelector() ও querySelectorAll() — CSS সিলেক্টর দিয়ে
- ▶ HTMLCollection (live) বনাম NodeList (static) — কেন এই পার্থক্য বাগ তৈরি করে
- ▶ NodeList-কে আয়েতে নেওয়া: [...nodeList] বা Array.from()
- ▶ চলাচল: parentElement, children, firstElementChild, nextElementSibling, closest(), matches()
- ▶ document object: title, URL, body, head, documentElement, readyState
- ▶ স্ক্রিপ্ট কখন চালাবে: DOMContentLoaded বনাম load বনাম defer
- ▶ কর্মক্ষমতা — বারবার query না করে এলিমেন্ট ভেরিয়েবলে ক্যাশ করা

প্র্যাকটিস টাস্ক

১. একটি পেজে ৫টি ভিন্ন উপায়ে একই এলিমেন্ট নির্বাচন করে কনসোলে ছাপাও।
২. getElementsByClassName ও querySelectorAll দিয়ে নির্বাচন করে নতুন এলিমেন্ট যোগ করার পর দুটির length তুলনা করো।
৩. querySelectorAll দিয়ে সব নিয়ে forEach-এ ইনডেক্সসহ ছাপাও।
৪. closest() দিয়ে ক্লিক করা এলিমেন্টের নিকটতম কার্ড খুঁজে বের করো।
৫. DOM ট্রি ধরে parent → child → sibling — তিন দিকেই চলাচল করে দেখাও।
৬. document.title, document.URL ও document.readyState ছাপাও এবং title বদলাও।
৭. স্ক্রিপ্ট <head>-এ রেখে DOMContentLoaded ছাড়া ও সহ চালিয়ে পার্থক্য দেখো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: DOM Explorer Tool

যেকোনো পেজের গঠন বিশ্লেষণ করার একটি টুল — কোন ট্যাগ কতবার আছে, কত গভীর এবং কোন এলিমেন্টে কী ক্লাস।

- ▶ ☐ পেজের সব এলিমেন্ট গুনে ট্যাগ-ভিত্তিক পরিসংখ্যান টেবিল

- ▶ ☐ সবচেয়ে গভীর nesting স্তর ও সেই পথ (breadcrumb) বের করা
- ▶ ☐ সিলেক্টর ইনপুট বক্স — লিখলে কতগুলো এলিমেন্ট মেলে তা লাইভ দেখানো
- ▶ ☐ ছবির alt নেই বা লিংকের টেক্সট খালি — এমন সমস্যা চিহ্নিত করা
- ▶ ☐ সব ফলাফল console.table() ও পেজে — দুই জায়গায় প্রদর্শন

দিন ২৭ · DOM পরিবর্তন, CSS ও Animation

আজকের টপিক

HTML DOM - Changing HTML · HTML DOM - Changing CSS · HTML DOM Animation

মূল কনসেপ্ট

- ▶ কনটেন্ট বদল: `textContent`, `innerText`, `innerHTML` — নিরাপত্তা (XSS) ও কর্মক্ষমতার পার্থক্য
- ▶ অ্যাট্রিবিউট: `getAttribute`, `setAttribute`, `removeAttribute`, `hasAttribute`, `dataset`
- ▶ এলিমেন্ট তৈরি ও বসানো: `createElement`, `append`, `prepend`, `before`, `after`, `insertAdjacentHTML`, `remove`, `replaceWith`
- ▶ `DocumentFragment` দিয়ে একসাথে অনেক এলিমেন্ট যোগ করে `reflow` কমানো
- ▶ স্টাইল বদল: `element.style (inline)` বনাম `classList.add/remove/toggle/contains/replace`
- ▶ CSS ভেরিয়েবল বদল: `style.setProperty('--main, 'red')`
- ▶ `getComputedStyle()` — প্রকৃতপক্ষে প্রযোজ্য স্টাইল পড়া
- ▶ অ্যানিমেশন: `setInterval`-ভিত্তিক বনাম `requestAnimationFrame`
- ▶ CSS transition ও `@keyframes` ক্লাস টগল করে চালানো — সবচেয়ে দক্ষ উপায়
- ▶ Web Animations API (`element.animate()`) সংক্ষেপে
- ▶ Reflow ও repaint — কেন বারবার DOM ছোঁয়া ব্যয়বহুল

প্র্যাকটিস টাস্ক

১. একই লেখা `textContent` ও `innerHTML` দিয়ে বসিয়ে ট্যাগসহ পার্থক্য দেখাও।
২. `<script>` ট্যাগসহ ইনপুট `innerHTML`-এ বসিয়ে XSS ঝুঁকি বুঝে `textContent`-এ রূপান্তর করো।
৩. লুপে ১০০০টি `<li>` তৈরি করো — একবার সরাসরি, একবার `DocumentFragment` দিয়ে; সময় তুলনা করো।
৪. `classList.toggle()` দিয়ে ডার্ক ও লাইট থিম বদলাও।
৫. CSS ভেরিয়েবল বদলে পুরো পেজের রঙ-স্কিম পাল্টাও।
৬. `requestAnimationFrame` দিয়ে একটি বক্স বাঁ থেকে ডানে সরো।
৭. `dataset` দিয়ে প্রতিটি কার্ডে `data-id` বসিয়ে পরে সেটি পড়ো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Dynamic Card Builder + Theme Switcher

ডেটা অ্যারে থেকে কার্ড তৈরি, সম্পাদনা ও মুছে ফেলার একটি ইন্টারফেস, সাথে থিম ও অ্যানিমেশন।

- ▶ ☐ অ্যারের ডেটা থেকে `createElement` ও `DocumentFragment` দিয়ে কার্ড রেন্ডার

- ▶ ☐ প্রতিটি কার্ডে data-id; সম্পাদনা করলে textContent আপডেট
- ▶ ☐ ডার্ক/লাইট থিম টগল — classList ও CSS ভেরিয়েবল দিয়ে
- ▶ ☐ নতুন কার্ড যোগ হলে fade-in, মুছেলে slide-out অ্যানিমেশন
- ▶ ☐ একটি প্রোগ্রেস বার requestAnimationFrame দিয়ে চলমান

দিন ২৮ · Events — Listener, Bubbling ও Delegation

আজকের টপিক

JavaScript Events · JavaScript EventListener · JavaScript Event Management · JavaScript Mouse Events · JavaScript Keyboard Events · JavaScript Load Events · JavaScript HTML DOM Events

মূল কনসেপ্ট

- ▶ ইভেন্ট যুক্ত করার ৩ উপায়: HTML অ্যাট্রিবিউট, onclick property, addEventListener (সেরা)
- ▶ addEventListener(type, handler, options) — once, capture, passive; এবং removeEventListener
- ▶ Event object: target, currentTarget, type, preventDefault(), stopPropagation()
- ▶ Bubbling ও capturing — ইভেন্ট কোন পথে ওঠানামা করে
- ▶ Event delegation — অভিভাবকে একটি লিসেনার দিয়ে অনেক সন্তান সামলানো
- ▶ Mouse: click, dblclick, mousedown/up, mouseenter/leave, mouseover/out, mousemove, contextmenu
- ▶ Keyboard: keydown, keyup; event.key, event.code, ctrlKey, shiftKey, altKey
- ▶ Form ও input: input, change, submit, focus, blur
- ▶ Load: DOMContentLoaded, load, beforeunload, resize, scroll
- ▶ Debounce ও throttle — scroll ও input ইভেন্টে কর্মক্ষমতা বাঁচানো

প্র্যাকটিস টাস্ক

১. একই বোতামে তিনভাবে ইভেন্ট যুক্ত করে পার্থক্য ও সীমাবদ্ধতা লেখো।
২. Bubbling দেখাতে nested তিনটি div-এ লিসেনার দাও, তারপর stopPropagation() দিয়ে থামাও।
৩. Event delegation দিয়ে একটি তালিকার যেকোনো আইটেমে ক্লিক ধরো — নতুন আইটেমেও যেন কাজ করে।
৪. কীবোর্ড শর্টকাট বানাও: Ctrl + S চাপলে "সেভ হলো" (ব্রাউজারের ডিফল্ট আচরণ বন্ধ করে)।
৫. mousemove দিয়ে কার্সরের স্থানাঙ্ক লাইভ দেখাও।
৬. input ইভেন্টে debounce লাগিয়ে সার্চ কলের সংখ্যা কমাও।
৭. beforeunload দিয়ে অসম্পূর্ণ ফর্ম রেখে চলে যেতে চাইলে সতর্ক করো।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Interactive To-Do App (সপ্তাহ ৪ ক্যাপস্টোন)

সম্পূর্ণ ইভেন্ট-চালিত একটি টু-ডু অ্যাপ — যোগ, সম্পন্ন, সম্পাদনা, মুছে ফেলা, ছাঁকা ও কীবোর্ড শর্টকাট।

- ▶ ☐ ফর্মের submit ইভেন্টে নতুন কাজ যোগ; preventDefault() দিয়ে রিলোড বন্ধ

- ▶ ☐ Event delegation দিয়ে একটি লিসেনারেই সম্পন্ন, সম্পাদনা ও মুছে ফেলা
- ▶ ☐ ফিল্টার বোতাম (সব / বাকি / সম্পন্ন) এবং লাইভ সার্চ debounce সহ
- ▶ ☐ কীবোর্ড শর্টকাট: Enter দিয়ে যোগ, Escape দিয়ে বাতিল, Ctrl+Enter দিয়ে সব সম্পন্ন
- ▶ ☐ মোট, বাকি ও সম্পন্ন কাজের গণনা এবং beforeunload সতর্কতা

শেষ ধাপ · দিন ২৯-৩০

ডিবাগিং, HTML-First ও ক্যাপস্টোন

দিন ২৯ · Errors, Debugging ও Best Practices

আজকের টপিক

JavaScript Errors · JavaScript Silent Errors · JavaScript Error Statement · JavaScript Error Reference · JavaScript Debugging · Debugging Console · Debugging Breakpoints · Debugging Errors · Debugging Async · JavaScript Style Guide · JavaScript Best Practices · JavaScript Common Mistakes · JavaScript Performance

মূল কনসেপ্ট

- ▶ Error-এর ধরন: `SyntaxError`, `ReferenceError`, `TypeError`, `RangeError`, `URIError`, `EvalError`
- ▶ `try...catch...finally` এবং `throw` দিয়ে নিজের error তৈরি
- ▶ Error object: `name`, `message`, `stack`; এবং custom error class বানানো
- ▶ Silent error — খালি `catch {}`, `??` ও `?`. দিয়ে সমস্যা ঢেকে ফেলা, ভুল ডেটা নীরবে বয়ে চলা
- ▶ Console টুল: `log`, `table`, `dir`, `group`, `time/timeEnd`, `count`, `assert`, `trace`
- ▶ DevTools breakpoint: `line`, `conditional`, `DOM change` এবং `event listener breakpoint`
- ▶ `Step over` / `step into` / `step out`; `watch`, `scope` ও `call stack` প্যানেল
- ▶ debugger স্টেটমেন্ট দিয়ে কোডেই থামানো
- ▶ Async debugging — `setTimeout` ও `Promise`-এর `stack trace`, DevTools-এর Async অপশন
- ▶ Style guide: নামকরণ, ইন্ডেন্ট, `===`, `const` আগে, ম্যাজিক নম্বর এড়ানো
- ▶ সাধারণ ভুল: `=` বনাম `==`, ভাসমান দশমিক তুলনা, `this` হারানো, লুপের ভেতরে DOM ছোঁয়া
- ▶ Performance: DOM ক্যাশ, লুপে কাজ কমানো, `debounce/throttle`, অপ্রয়োজনীয় গ্লোবাল বাদ

প্র্যাকটিস টাস্ক

১. ছয় ধরনের error ইচ্ছাকৃতভাবে ঘটিয়ে প্রতিটির বার্তা ও stack পড়ো।
২. `try...catch...finally` দিয়ে JSON পার্স করো — ভুল JSON-এ বন্ধুত্বপূর্ণ বার্তা দাও।
৩. একটি custom `ValidationError` ক্লাস বানিয়ে `throw` করো।
৪. খালি `catch {}` লিখে একটি বাগ লুকাও, তারপর সঠিকভাবে হ্যান্ডেল করো।
৫. `console.time/timeEnd` ও `console.table` দিয়ে দুটি অ্যালগরিদমের তুলনা করো।

৬. DevTools-এ conditional breakpoint বসিয়ে লুপের ৫০তম চক্রে থামো।

৭. আগের যেকোনো একটি প্রজেক্ট স্টাইল-গাইড মেনে রিফ্যাক্টর করো — নামকরণ, ম্যাজিক নম্বর ও ফাংশনের দৈর্ঘ্য।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: Bug Hunt & Code Quality Audit

নিজের আগের প্রজেক্টগুলোর ওপর একটি পূর্ণ অডিট — বাগ ধরা, error handling যোগ করা এবং কর্মক্ষমতা মাপা।

- ▶ ☐ অন্তত ৩টি পুরনো প্রজেক্টে try...catch ও ইনপুট যাচাই যোগ করা
- ▶ ☐ একটি কেন্দ্রীয় logger.js — info/warn/error, console.group দিয়ে সাজানো আউটপুট
- ▶ ☐ ইচ্ছাকৃতভাবে ১০টি বাগসহ একটি ফাইল বানিয়ে সেগুলো খুঁজে সমাধানের নোট লেখা
- ▶ ☐ console.time দিয়ে রিফ্যাক্টরের আগে-পরে কর্মক্ষমতার তুলনা টেবিল
- ▶ ☐ README-তে নিজের "কোড স্টাইল গাইড" ও "১৫টি সাধারণ ভুল" তালিকা

আজকের টপিক

HTML Form Validation · HTML-First Tutorial · Progressive Enhancement · Native HTML Features · CSS Instead of JavaScript

মূল কনসেপ্ট

- ▶ HTML-এর নিজস্ব যাচাই: required, type="email", min/max, minlength/maxlength, pattern, step
- ▶ Constraint Validation API: checkValidity(), reportValidity(), setCustomValidity(), validity object
- ▶ :valid, :invalid ও :user-invalid দিয়ে CSS-এ ফিডব্যাক
- ▶ novalidate দিয়ে নিজের যাচাই চালানো — কখন সেটি দরকার
- ▶ HTML-First চিন্তা: আগে সেমান্টিক HTML, তারপর CSS, সবশেষে JS
- ▶ Progressive enhancement — JS বন্ধ থাকলেও মূল কাজ যেন চলে
- ▶ Native HTML: <details>, <dialog>, <datalist>, <progress>, <meter>, <template>, popover, input type=date/color/range
- ▶ CSS দিয়েই যা হয়: hover/focus, transition, :has(), scroll-snap, sticky, অ্যাকর্ডিয়ন, মোডাল
- ▶ অ্যাকসেসিবিলিটি: label, aria-*, কীবোর্ড নেভিগেশন, ফোকাস ব্যবস্থাপনা
- ▶ কম JS মানে কম বাগ, দ্রুত পেজ ও কম রক্ষণাবেক্ষণ

প্র্যাকটিস টাস্ক

১. একটি ফর্ম শুধু HTML অ্যাট্রিবিউট দিয়ে যাচাই করাও — এক লাইনও JS নয়।
২. setCustomValidity() দিয়ে বাংলা ভাষায় নিজের ত্রুটি-বার্তা দাও।
৩. :invalid ও :user-invalid দিয়ে CSS-এ লাল বর্ডার দেখাও।
৪. <details>, <dialog> ও <datalist> — তিনটিই ব্যবহার করে JS ছাড়া UI বানাও।
৫. একটি অ্যাকর্ডিয়ন ও একটি ট্যাব শুধু CSS দিয়ে বানাও।
৬. তোমার To-Do অ্যাপের JS বন্ধ করে দেখো কতটা কাজ করে; মূল কাজটুকু HTML দিয়ে চালু রাখো।
৭. :has() ব্যবহার করে অভিভাবক এলিমেন্ট স্টাইল করো — আগে যা কেবল JS দিয়ে সম্ভব ছিল।

আজকের রিয়েল প্রজেক্ট

প্রজেক্ট: ফাইনাল ক্যাপস্টোন — Personal Learning Dashboard

পুরো মাসের সব শেখা এক প্রজেক্টে — একটি ব্যক্তিগত ড্যাশবোর্ড যেখানে কোর্স, কাজ, খরচ, সময়সূচি ও পরিসংখ্যান একসাথে।

- ▶ ☐ সেমান্টিক HTML ও native ফর্ম যাচাই দিয়ে সব ইনপুট — JS বন্ধ থাকলেও ফর্ম কাজ করে
- ▶ ☐ Object/Array/Map/Set দিয়ে ডেটা স্তর এবং reduce দিয়ে সব পরিসংখ্যান
- ▶ ☐ Temporal বা Date দিয়ে সময়সূচি ও কাউন্টডাউন, RegExp দিয়ে ইনপুট যাচাই
- ▶ ☐ Event delegation, debounce, classList থিম ও requestAnimationFrame অ্যানিমেশন
- ▶ ☐ সম্পূর্ণ error handling, কমেন্ট, README, GitHub Pages-এ ডিপ্লয় ও পোর্টফোলিওতে যুক্ত করা

৩০ দিনের পর — পরবর্তী ধাপ

এই রুটিনের ৩০ দিনে JavaScript-এর মূল ভাষা ও DOM সম্পূর্ণ হয়ে যাবে। এরপর যে বিষয়গুলো ধরলে তুমি সরাসরি প্রফেশনাল কাজের জন্য প্রস্তুত হবে:

- ▶ Asynchronous JavaScript — callback, Promise, async/await, event loop গভীরে
- ▶ Fetch API ও REST API — সার্ভার থেকে ডেটা এনে DOM-এ দেখানো, error ও loading state সামলানো
- ▶ ES6 Modules — import/export দিয়ে কোড ফাইলে ফাইলে ভাগ করা
- ▶ Classes ও OOP — class, extends, super, static, private field (#)
- ▶ Web Storage — localStorage, sessionStorage, IndexedDB দিয়ে ডেটা ধরে রাখা
- ▶ Browser API — Fetch, History, Geolocation, Intersection Observer, Clipboard
- ▶ Build tools ও npm — Vite, ESLint, Prettier দিয়ে পেশাদার সেটআপ
- ▶ একটি ফ্রেমওয়ার্ক — এই ভিত্তি নিয়ে React বা অন্য যেকোনোটিতে ঢোকা অনেক সহজ হবে

GitHub ও পোর্টফোলিও রুটিন

- ▶ একটি রিপো: js-30-days — ভেতরে day-01 থেকে day-30 পর্যন্ত ৩০টি ফোল্ডার।
- ▶ প্রতিটি ফোল্ডারে index.html, app.js এবং একটি ছোট README.md (কী শিখলাম, কী বানালাম)।
- ▶ প্রতিদিন অন্তত একটি অর্থবহ কমিট — যেমন: day-12: closure-based wallet module।
- ▶ সপ্তাহ শেষে ক্যাপস্টোন প্রজেক্টগুলো GitHub Pages-এ ডিপ্লয় করে লাইভ লিংক README-তে যোগ করো।
- ▶ মূল README-তে ৩০ দিনের একটি চেকলিস্ট রাখো — শেষ হওয়া দিন টিক দিয়ে অগ্রগতি দৃশ্যমান রাখো।

প্রতিদিন এক ধাপ। ৩০ দিন পর পেছনে তাকালে দূরত্বটা নিজেই অবাক করবে।